UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
THÀNH PHỐ HỒ CHÍ MINH
BAN CÔNG NGHỆ SỐ

ĐIỆN TOÁN ĐÁM MÂY

Chương 2 – Mô hình dịch vụ và hạ tầng điện toán đám mây

Số tín chỉ: **3**

Giảng viên: TS. Lê Quốc Tuấn
Email : Tuan.le@ut.edu.vn

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI THÀNH PHỐ HỒ CHÍ MINH



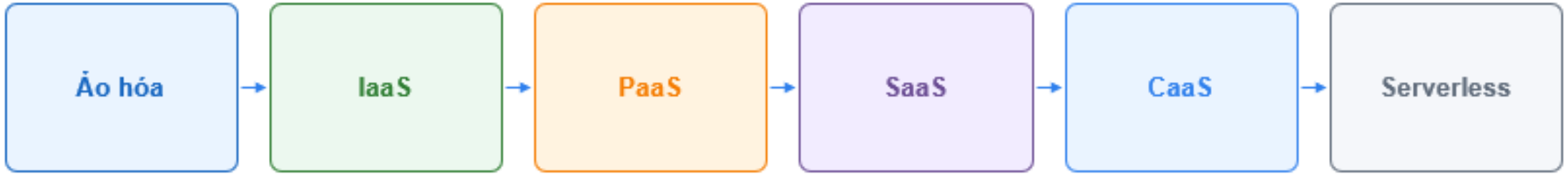
Mục tiêu học tập

- Giải thích vai trò của ảo hóa trong điện toán đám mây.
- Mô tả các thành phần tính toán, mạng và lưu trữ của IaaS.
- Phân biệt IaaS, PaaS, SaaS, CaaS và serverless.
- Lựa chọn mô hình dịch vụ phù hợp với từng bài toán triển khai.
- Thực hiện được các bước thực hành cơ bản: máy ảo, mạng, lưu trữ, ứng dụng.



Mục tiêu học tập

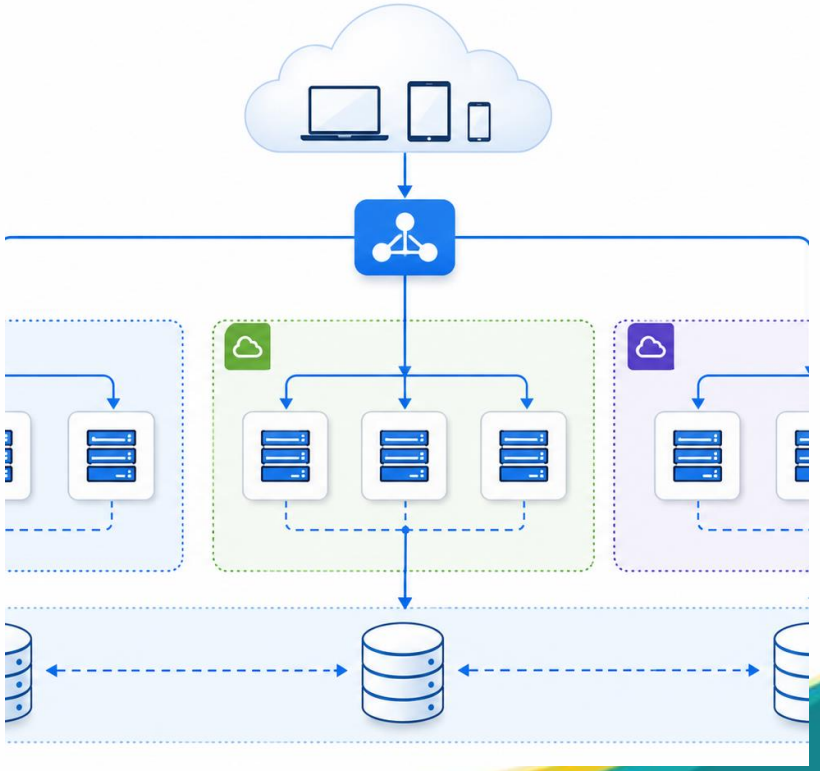
Bức tranh tổng quan Chương 2



- Ảo hóa là nền tảng kỹ thuật giúp chia sẻ tài nguyên vật lý.
- IaaS cung cấp hạ tầng; PaaS cung cấp nền tảng; SaaS cung cấp phần mềm hoàn chỉnh.
- CaaS (Container-as-a-Service) và serverless (Điện toán phi máy chủ) là các mô hình hiện đại hỗ trợ cloud-native.

Tình huống: triển khai hệ thống Web

- Doanh nghiệp cần triển khai website có lượng truy cập biến động mạnh.
- Cần máy chủ Web, cơ sở dữ liệu, mạng, lưu trữ, bảo mật và khả năng mở rộng.
- Câu hỏi: nên thuê máy ảo, dùng nền tảng ứng dụng, dùng phần mềm SaaS hay serverless?



Phần 1. Công nghệ ảo hóa

- Ảo hóa giúp nhiều môi trường độc lập cùng dùng chung một hạ tầng vật lý.



Ảo hóa là gì?

- Ảo hóa tạo ra phiên bản logic của tài nguyên vật lý: máy chủ, mạng, lưu trữ.
- Mỗi máy ảo có hệ điều hành, ứng dụng và cấu hình riêng.
- Nhiều máy ảo có thể chạy đồng thời trên cùng một máy chủ vật lý.
- Ảo hóa là một công nghệ nền; điện toán đám mây là mô hình dịch vụ rộng hơn.

Ảo hóa


Các loại ảo hóa phổ biến
1. Ảo Hóa Máy Chủ (Server Virtualization)
2. Ảo Hóa Lưu Trữ (Storage Virtualization)
3. Ảo Hóa Mạng (Network Virtualization)
4. Ảo Hóa Desktop (Virtual Desktop Infrastructure - VDI)

Vì sao ảo hóa quan trọng trong cloud?

- Tăng hiệu suất sử dụng tài nguyên vật lý.
- Cấp phát máy chủ nhanh hơn so với mua sắm phần cứng.
- Cô lập môi trường giữa các khách hàng hoặc nhóm ứng dụng.
- Hỗ trợ snapshot, template, migration và phục hồi nhanh.
- Tạo nền tảng cho tự động hóa và co giãn tài nguyên.

Mục tiêu: biến phần cứng cố định thành tài nguyên linh hoạt

Kết quả: hạ tầng có thể cấp phát như một dịch vụ

Hypervisor: lớp quản lý máy ảo

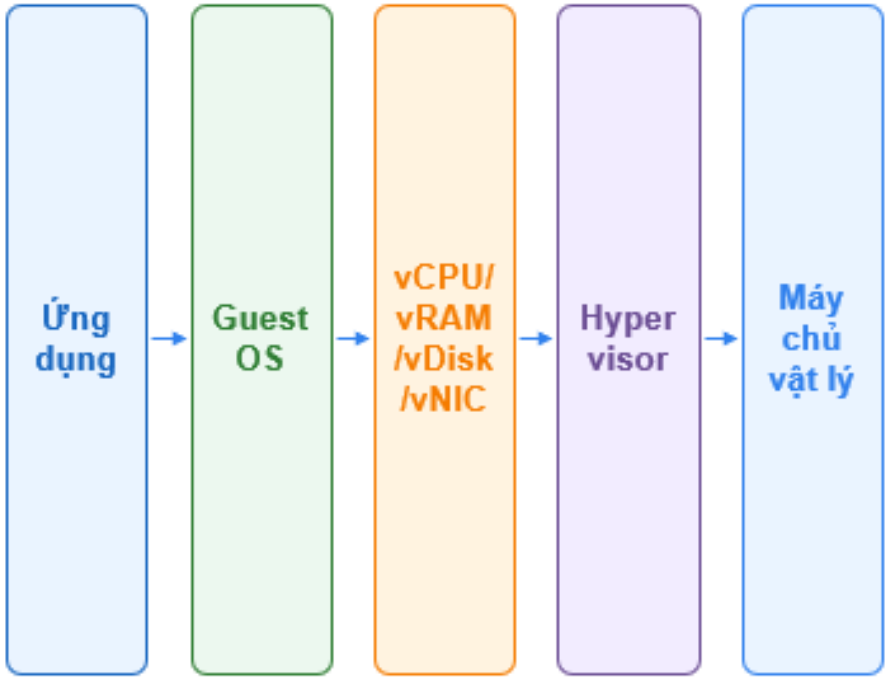
- **Hypervisor** (hay còn gọi là trình giám sát máy ảo - VMM) là lớp phần mềm hoặc phần cứng cốt lõi cho phép tạo, chạy và quản lý các máy ảo (VM). Nó hoạt động như một lớp trung gian, phân bổ tài nguyên vật lý của máy chủ (CPU, RAM, bộ nhớ) cho từng máy ảo một cách độc lập và hiệu quả.
- Hypervisor được chia làm hai loại chính với đặc điểm và mục đích sử dụng khác nhau:

Hypervisor: lớp quản lý máy ảo

- **Hypervisor Loại 1 (Bare-Metal):** Cài đặt trực tiếp lên phần cứng vật lý (thay thế hệ điều hành). Nó có hiệu suất tối đa, bảo mật cao và thường dùng cho các hệ thống máy chủ doanh nghiệp. Ví dụ: VMware ESXi, Microsoft Hyper-V, KVM.
- **Hypervisor Loại 2 (Hosted):** Chạy như một ứng dụng phần mềm trên một Hệ điều hành nền tảng (như Windows, macOS hoặc Linux). Thích hợp cho việc thử nghiệm, phát triển phần mềm hoặc sử dụng cá nhân. Ví dụ: Oracle VM VirtualBox, VMware Workstation.

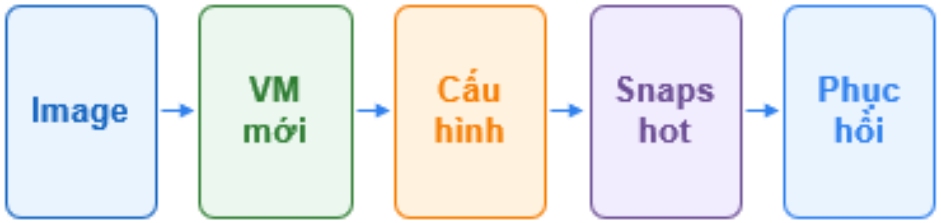
Cấu trúc của một máy ảo

- vCPU: phần CPU được cấp cho máy ảo.
- vRAM: dung lượng bộ nhớ logic.
- Virtual disk: đĩa ảo lưu hệ điều hành và dữ liệu.
- Virtual NIC: giao diện mạng ảo kết nối vào mạng cloud.
- Guest OS: hệ điều hành chạy trong máy ảo.



Ảnh máy ảo, template và snapshot

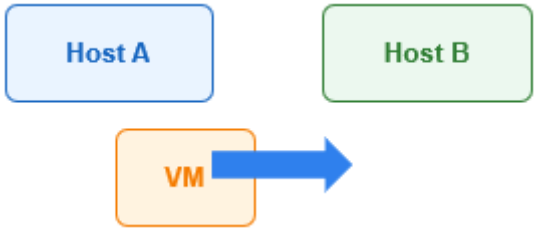
- Image: ảnh hệ điều hành hoặc ảnh máy đã chuẩn hóa.
- Template: khuôn mẫu dùng để tạo nhanh nhiều máy giống nhau.
- Snapshot: bản chụp trạng thái đĩa tại một thời điểm.
- Các cơ chế này giúp triển khai, sao lưu và phục hồi nhanh hơn.



- **Lưu ý: snapshot không thay thế hoàn toàn chiến lược backup.**
- **Cần quản lý vòng đời snapshot để tránh tăng chi phí.**

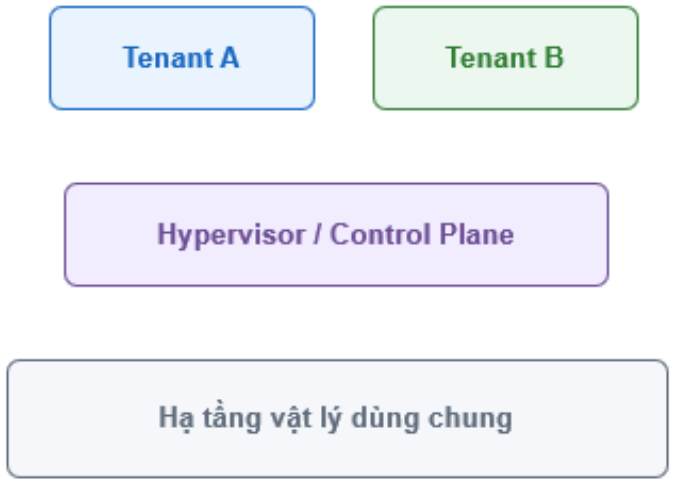
Live migration và tính linh hoạt hạ tầng

- Live migration cho phép di chuyển máy ảo giữa các máy chủ vật lý với thời gian gián đoạn rất thấp.
- Hữu ích khi bảo trì phần cứng hoặc cân bằng tải tài nguyên.
- Cloud provider dùng nhiều kỹ thuật điều phối để giảm gián đoạn cho khách hàng.
- Không phải mọi dịch vụ đều hỗ trợ migration theo cùng một cách.



Đa thuê và cô lập tài nguyên

- Multi-tenancy cho phép nhiều khách hàng dùng chung hạ tầng vật lý.
- Cô lập logic giúp dữ liệu, mạng và tài nguyên của từng khách hàng tách biệt.
- Cô lập có thể ở mức VM, mạng, lưu trữ, IAM và dịch vụ ứng dụng.
- Thiết kế cô lập là điều kiện quan trọng để bảo mật cloud.



Phần 2. Dịch vụ hạ tầng - IaaS

- IaaS cung cấp tài nguyên hạ tầng: tính toán, mạng, lưu trữ và bảo mật cơ bản.
- **IaaS** (Cơ sở hạ tầng dưới dạng dịch vụ) là mô hình điện toán đám mây cho phép doanh nghiệp thuê các tài nguyên CNTT cơ bản như máy chủ, dung lượng lưu trữ và hệ thống mạng. Bạn có thể quản lý hệ thống của mình mà không cần mua phần cứng vật lý và chỉ thanh toán theo mức thực tế sử dụng (pay-as-you-go)


IaaS
INFRASTRUCTURE AS A SERVICE
AUTOMATED
DATA CENTERS
HARDWARE
NETWORK
LOAD BALANCERS
STORAGE
COMPUTE
CLOUD HOSTING
DYNAMIC

UTH UNIVERSITY OF TRANSPORT HOCHIMINH CITY

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI THÀNH PHỐ HỒ CHÍ MINH





IaaS là gì?

- Infrastructure as a Service cung cấp hạ tầng CNTT dưới dạng dịch vụ.
- Người dùng tự cấu hình máy ảo, mạng, đĩa, firewall và cân bằng tải.
- Nhà cung cấp quản lý trung tâm dữ liệu, phần cứng, ảo hóa và dịch vụ nền.
- IaaS phù hợp khi cần kiểm soát cao hệ điều hành và kiến trúc triển khai.



Các thành phần chính của IaaS



Compute (Lớp điện toán)

- Lớp chịu trách nhiệm xử lý logic, tính toán và chạy các ứng dụng.
- **Máy ảo (Virtual Machine - VM):** Phiên bản phần cứng ảo hóa chạy trên hạ tầng vật lý (thông qua Hypervisor như KVM, ESXi, Hyper-V). Cung cấp CPU ảo (vCPU), RAM và hệ điều hành riêng biệt.
- **Image (Machine Image):** Mẫu định dạng sẵn chứa hệ điều hành, cấu hình phần mềm và dữ liệu ban đầu dùng để khởi tạo máy ảo nhanh chóng và đồng nhất.
- **Autoscaling:** Cơ chế tự động tăng hoặc giảm số lượng tài nguyên máy ảo dựa trên tải thực tế (CPU, RAM, Network Traffic) nhằm tối ưu chi phí và đảm bảo tính sẵn sàng (High Availability).

Phân tích chi tiết từng phân vùng kiến trúc - IaaS

Network (Lớp mạng)

- Lớp thiết lập môi trường kết nối, cô lập tài nguyên và điều hướng lưu lượng.
- **VPC / VNet (Virtual Private Cloud / Virtual Network):** Mạng riêng ảo được cô lập logic trên đám mây công cộng, cho phép doanh nghiệp toàn quyền kiểm soát không gian địa chỉ IP và cấu hình mạng.
- **Subnet (Mạng con):** Phân đoạn không gian địa chỉ IP trong VPC/VNet (thường chia thành *Public Subnet* có kết nối Internet và *Private Subnet* nội bộ).
- **Routing (Route Table):** Tập hợp các quy tắc (rules) xác định hướng đi của lưu lượng mạng từ Subnet hoặc Gateway đến các đích khác.
- **Gateway:** Cổng nối cho phép giao tiếp giữa mạng nội bộ VPC/VNet với các mạng bên ngoài (Internet Gateway, NAT Gateway, VPN Gateway).
- **Load Balancer (Bộ cân bằng tải):** Thiết bị/dịch vụ phân phối lưu lượng truy cập từ người dùng đến nhiều máy ảo phía sau để tránh quá tải và tăng độ tin cậy.

Storage (Lớp lưu trữ)

Lớp lưu trữ dữ liệu với các thuộc tính và hiệu năng khác nhau tùy thuộc vào bài toán sử dụng.

- **Block Storage (Lưu trữ khối):** Dữ liệu chia thành các khối cố định, gắn trực tiếp vào máy ảo như một ổ cứng vật lý (e.g., SSD/HDD). Phù hợp cho CSDL, Hệ điều hành cần tốc độ I/O cao.
- **File Storage (Lưu trữ tệp):** Hệ thống lưu trữ chia sẻ cấp tệp (Network File System - NFS/SMB), cho phép nhiều máy ảo cùng truy cập và chia sẻ tệp dữ liệu đồng thời.
- **Object Storage (Lưu trữ đối tượng):** Lưu trữ dữ liệu phi cấu trúc (ảnh, video, tệp sao lưu) dưới dạng "đối tượng" đi kèm Metadata và ID duy nhất, truy cập thông qua RESTful API/HTTP/HTTPS. Khả năng mở rộng gần như vô hạn.
- **Snapshot:** Bản ghi trạng thái chỉ đọc (Read-Only) của một đĩa lưu trữ tại một thời điểm nhất định, dùng để khôi phục hoặc tạo đĩa mới.
- **Backup:** Quy trình tự động hoặc thủ công tạo bản sao lưu toàn bộ hệ thống/dữ liệu theo lịch trình để phòng chống rủi ro thảm họa (Disaster Recovery).

Security (Lớp an toàn & An ninh thông tin)

Lớp kiểm soát truy cập và bảo vệ tài nguyên khỏi các hiểm họa mạng.

- **Security Group (SG):** Tường lửa ảo ở cấp độ **thể hiện máy ảo (Instance level)**, kiểm soát lưu lượng vào/ra (Stateful - tự động mở chiều phản hồi).
- **ACL (Network Access Control List):** Tường lửa ảo ở cấp độ **Subnet level**, đóng vai trò là lớp bảo vệ bổ sung (Stateless - phải khai báo thủ công cả chiều vào và ra).
- **Key Pair:** Cặp khóa bất đối xứng (Public Key & Private Key) dùng để xác thực an toàn khi truy cập từ xa vào máy ảo (SSH cho Linux, RDP cho Windows).
- **IAM (Identity and Access Management):** Hệ thống quản lý danh tính và quyền hạn, cho phép phân quyền chi tiết (RBAC) cho người dùng, nhóm hoặc dịch vụ được phép thao tác trên tài nguyên cloud.

Monitoring (Lớp giám sát & Vận hành)

Lớp thu thập dữ liệu vận hành để duy trì sức khỏe hệ thống và ứng phó sự cố.

- **Log:** Bản ghi chi tiết các sự kiện, nhật ký hoạt động của hệ thống, hệ điều hành, dịch vụ hoặc ứng dụng diễn ra theo thời gian.
- **Metric:** Các chỉ số đo lường định lượng hiệu năng hệ thống theo chu kỳ (như % CPU utilization, Memory usage, Disk Read/Write, Network In/Out).
- **Alert:** Cơ chế cảnh báo tự động gửi thông báo (Email, SMS, Webhook) cho quản trị viên khi một hoặc nhiều Metric vượt ngưỡng an toàn đã thiết lập.

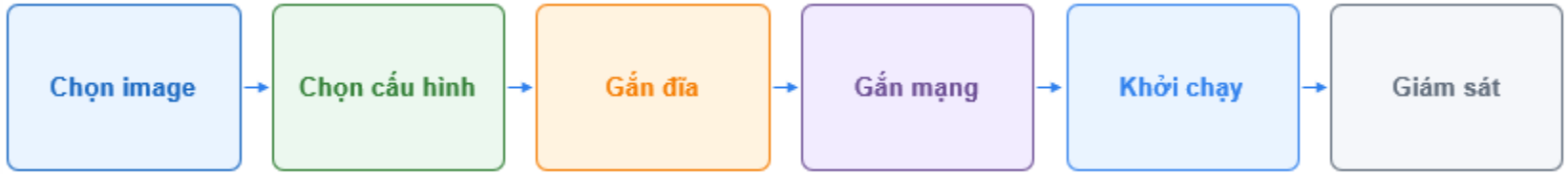
Bảng đối chiếu thuật ngữ tương đương trên các Cloud Provider hàng đầu

Thành phần	AWS (Amazon Web Services)	Microsoft Azure	GCP (Google Cloud)
Máy ảo	EC2 (Elastic Compute Cloud)	Azure Virtual Machines	Compute Engine
Image	AMI (Amazon Machine Image)	Azure VM Image	Compute Engine Image
Autoscaling	EC2 Auto Scaling	Virtual Machine Scale Sets (VMSS)	Compute Engine Autoscaling
VPC / VNet	Amazon VPC	Azure Virtual Network	VPC Network
Load Balancer	AWS ALB / NLB	Azure Load Balancer / App Gateway	Cloud Load Balancing
Block Storage	Amazon EBS	Azure Managed Disks	Persistent Disk
File Storage	Amazon EFS / FSx	Azure Files	Filestore
Object Storage	Amazon S3	Azure Blob Storage	Cloud Storage
Security Group	Security Group	Network Security Group (NSG)	Firewall Rules (Stateful)
ACL	Network ACL (NACL)	Subnet-level NSG	Cloud Armor / VPC Firewall
IAM	AWS IAM	Microsoft Entra ID (Azure AD)	Cloud IAM
Monitoring	Amazon CloudWatch	Azure Monitor	Google Cloud Observability

Dịch vụ tính toán trong IaaS

- Máy ảo là đơn vị tính toán phổ biến nhất trong IaaS.
- Mỗi máy có cấu hình CPU, RAM, hệ điều hành, đĩa và mạng.
- Có thể chọn kích thước máy theo nhu cầu: tổng quát, tối ưu CPU, tối ưu RAM, GPU.
- Có thể khởi tạo, tạm dừng, dừng, khởi động lại hoặc xóa máy.

Vòng đời một máy ảo



- Trước khi tạo VM cần xác định hệ điều hành, vùng triển khai, cấu hình và chính sách truy cập.
- Sau khi khởi chạy cần kiểm tra kết nối, cập nhật bản vá, cấu hình dịch vụ và bật giám sát.
- Khi không sử dụng cần dừng hoặc xóa tài nguyên để tránh phát sinh chi phí.

Lựa chọn cấu hình máy ảo

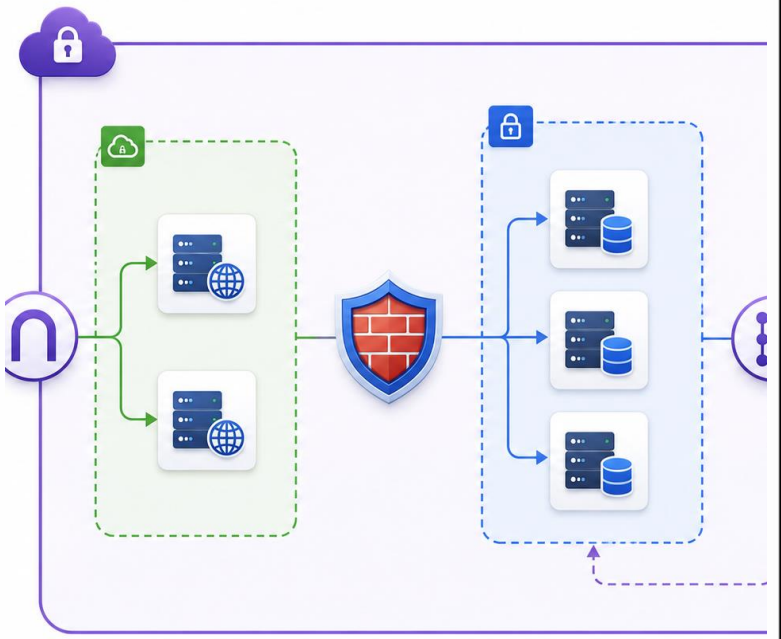
Nhu cầu	Cấu hình phù hợp	Ví dụ
Web nhỏ	General purpose	Website giới thiệu
Xử lý tính toán	Compute optimized	Tính toán batch
CSDL bộ nhớ cao	Memory optimized	Database/cache
AI/đồ họa	GPU/accelerator	Huấn luyện/suy luận AI

- Nguyên tắc: chọn vừa đủ, đo lường thực tế, sau đó tối ưu kích thước.
- Không nên chọn cấu hình quá lớn ngay từ đầu nếu chưa có dữ liệu tải.

Mạng đám mây trong IaaS

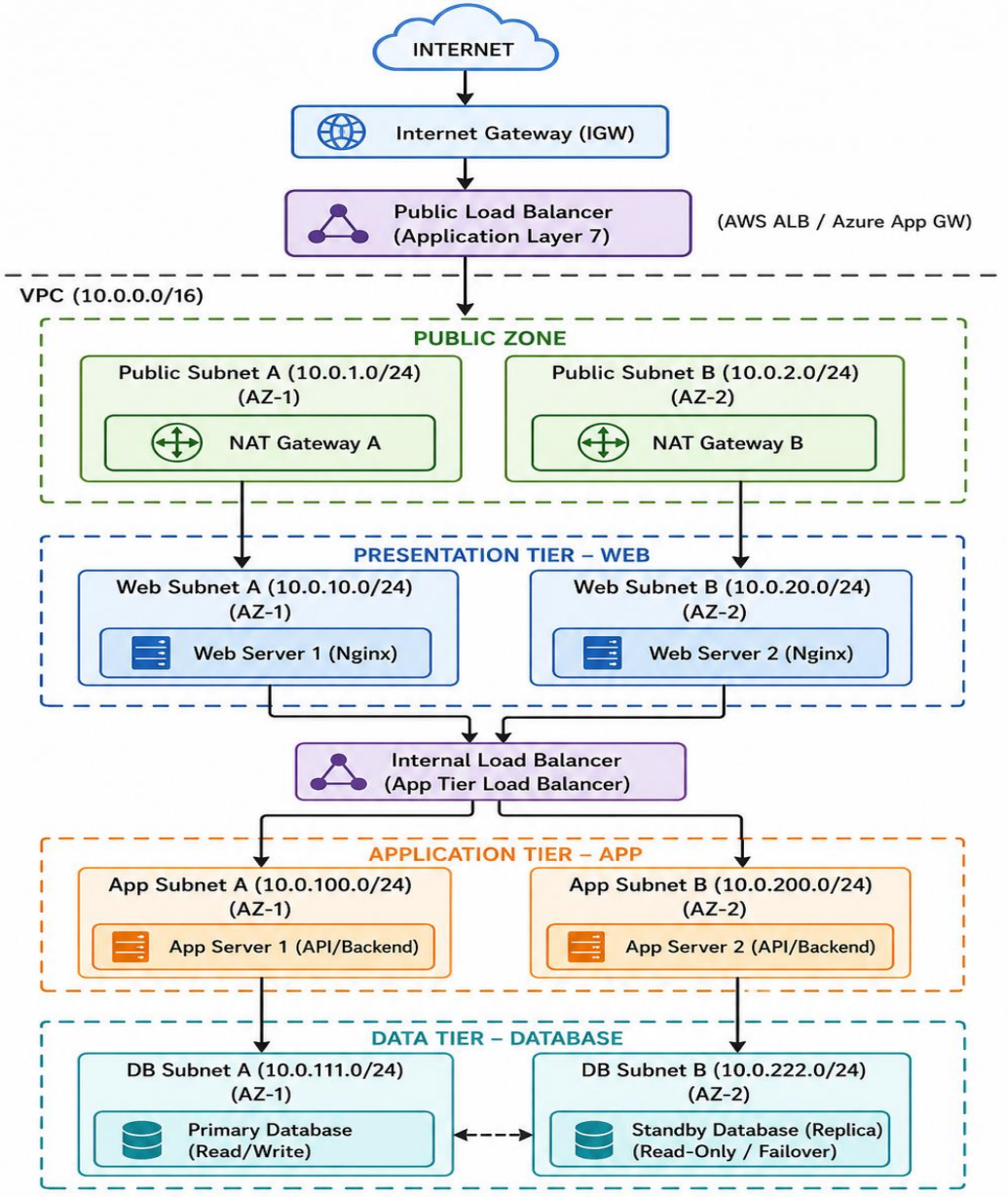
Mạng đám mây (Cloud networking) tạo mạng logic cô lập cho tài nguyên của người dùng.

- Các khái niệm cốt lõi: VPC/VNet, subnet, route table, gateway, firewall.
- Mạng đám mây quyết định cách máy ảo kết nối Internet, kết nối nội bộ và kết nối về doanh nghiệp.
- Thiết kế mạng đúng là nền tảng của bảo mật và khả năng mở rộng.



Kiến trúc Tổng thể (Architecture Diagram)

- Sơ đồ kiến trúc tổng thể và giải thích chi tiết cho một **mô hình Web 3 lớp (3-Tier Architecture) chuẩn High Availability (Tính sẵn sàng cao)** thiết kế trên 1 VPC trải dài qua **2 Availability Zones (AZ)** nhé.



Giải mã Thiết kế Kiến trúc (Architectural Breakdown)

Lớp 1: Public Zone & Ingress (Điểm tiếp nhận)

- **VPC CIDR:** 10.0.0.0/16 (Cung cấp tối đa 65,536 IP nội bộ).
- **Multi-AZ Strategy:** Triển khai song song trên 2 Vùng sẵn sàng (AZ-1 và AZ-2) độc lập về nguồn điện, mạng và phần cứng để chịu lỗi rủi ro thảm họa cấp Trung tâm dữ liệu (Datacenter Failover).
- **Internet Gateway (IGW):** Điểm giao tiếp 2 chiều duy nhất ra/vào Internet.
- **Public Load Balancer (ALB Layer 7):** Đứng ở Public Subnet, nhận HTTPS traffic từ Internet, thực hiện giải mã SSL/TLS Offloading và điều hướng luồng truy cập xuống tầng Web.

Lớp 2: Presentation Tier (Tầng Web)

- **Vị trí:** Đặt trong **Private Subnet** (không gán Public IP) để ẩn hoàn toàn khỏi Internet.
- **Chức năng:** Chạy các Web Server (Nginx, Apache) đóng vai trò xử lý giao diện, kiểm tra yêu cầu HTTP, caching nội dung tĩnh (static assets).
- **Outbound Traffic:** Muốn tải phần mềm hoặc cập nhật hệ điều hành, Web Server phải đi qua **NAT Gateway** đặt ở Public Subnet của AZ tương ứng.

Lớp 3: Application Tier (Tầng Ứng dụng/Logic)

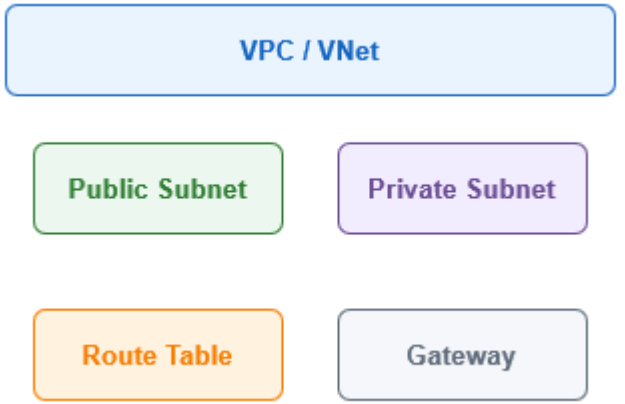
- **Internal Load Balancer:** Nằm giữa tầng Web và tầng App. Chỉ cho phép các Web Server truy xuất đến dịch vụ Backend.
- **Vị trí:** Đặt trong các **Private Subnet** riêng biệt.
- **Chức năng:** Chạy dịch vụ Backend/API (Node.js, Java Spring Boot, .NET Core, Python Fast-API) để thực thi business logic.
- **Autoscaling Group:** Cả Tầng Web và Tầng App đều được gán vào Autoscaling Group để tự động mở rộng/thu hẹp (Scale Out/In) số lượng máy ảo theo tải CPU/RAM.

Lớp 4: Data Tier (Tầng Dữ liệu)

- **Vị trí:** Mức bảo mật cao nhất, đặt tại **Isolated DB Subnet** không có đường routing đi ra Internet (không cấu hình NAT Gateway route).
- **Cấu hình High Availability (Multi-AZ Database):**
  - **Primary DB (AZ-1):** Nhận cả truy vấn Đọc và Ghi (Read/Write).
  - **Standby DB (AZ-2):** Đồng bộ dữ liệu liên tục theo cơ chế Synchronous Replication.
  - **Auto-Failover:** Khi AZ-1 gặp sự cố, hệ thống quản trị CSDL (như AWS RDS Multi-AZ) sẽ tự động nâng cấp Standby DB thành Primary và chuyển DNS Endpoint mà không gây gián đoạn ứng dụng.

VPC/VNet: mạng riêng ảo trên cloud

- VPC/VNet là không gian mạng logic do người dùng kiểm soát trong public cloud.
- Người dùng định nghĩa dải địa chỉ IP, subnet, tuyến đường và chính sách truy cập.
- Tài nguyên trong cùng VPC có thể giao tiếp nội bộ theo quy tắc mạng được cấu hình.
- VPC giúp cô lập môi trường dev, test, production hoặc từng hệ thống nghiệp vụ.

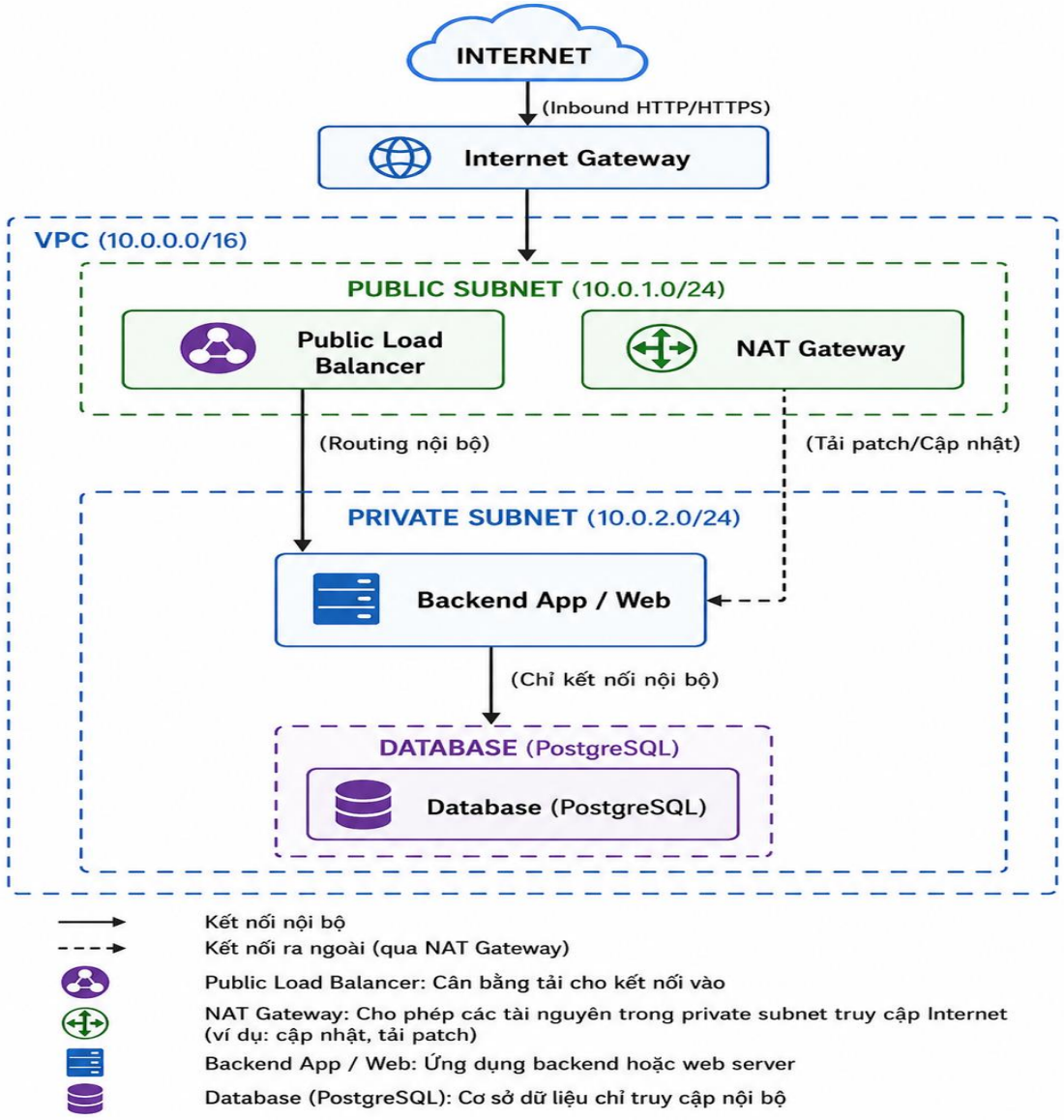


Subnet công cộng và subnet riêng

Tiêu chí	Public Subnet (Subnet công cộng)	Private Subnet (Subnet riêng)
Bản chất	Định tuyến trực tiếp ra/vào Internet.	Định tuyến nội bộ, hoàn toàn cô lập với Internet trực tiếp.
Internet Gateway (IGW)	**Có gắn** đường truyền tới Internet Gateway trong Bảng định tuyến (Route Table).	**Không gắn** Internet Gateway trong Route Table.
Loại địa chỉ IP	Các máy ảo nhận cả **Public IP** (hoặc Elastic IP) và Private IP.	Các máy ảo **chỉ nhận Private IP** nội bộ (ví dụ: 10.0.2.x).
Chiều kết nối Inbound *(Từ Internet truy cập vào)*	**Cho phép** kết nối trực tiếp từ Internet (qua IP công cộng hoặc Load Balancer).	**Chặn toàn bộ** kết nối chủ động từ Internet vào.
Chiều kết nối Outbound *(Từ máy ảo kết nối ra Internet)*	Kết nối trực tiếp ra Internet qua Internet Gateway.	Kết nối ra Internet **gián tiếp** thông qua **NAT Gateway** (chỉ dành cho các tác vụ xuất dữ liệu, tải bản vá).
Đối tượng triển khai	Web Server, Public Load Balancer, Bastion Host, API Gateway tiếp xúc người dùng.	Database (MySQL, PostgreSQL), Application Server (Backend API), Cache (Redis), File Storage.

Public & Private subnet

- **Luồng Inbound (Từ bên ngoài vào):**
  - Người dùng gửi yêu cầu từ Internet -> Đi qua **Internet Gateway** -> Đến **Public Load Balancer** nằm trong *Public Subnet*.
  - Load Balancer giải mã và điều hướng yêu cầu xuống các **Backend App** nằm an toàn bên trong *Private Subnet*.
- **Luồng Outbound (Từ bên trong đi ra):**
  - Máy chủ **Backend App** hoặc **Database** ở *Private Subnet* muốn cập nhật bản vá hệ điều hành.
  - Gói tin được gửi đến **NAT Gateway** (đặt ở *Public Subnet*). NAT Gateway thay thế IP nội bộ bằng IP công cộng của nó để gửi ra Internet, sau đó nhận kết quả và trả ngược lại cho máy chủ trong Private Subnet.
  - **Kết quả:** Máy chủ ở Private Subnet tải được dữ liệu từ Internet nhưng hacker bên ngoài không thể biết IP hay chủ động kết nối vào máy chủ này.



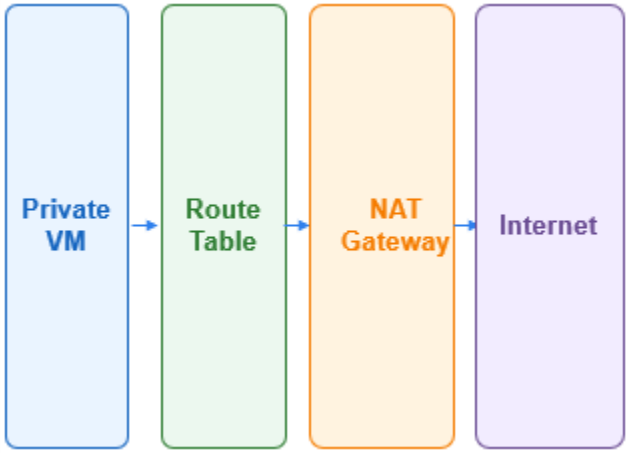
Địa chỉ IP công cộng và riêng

- IP riêng dùng cho giao tiếp nội bộ trong VPC/VNet.
- IP công cộng cho phép tài nguyên được định tuyến từ Internet.
- Không nên gán IP công cộng cho database hoặc máy chủ nội bộ.
- Nên dùng load balancer, NAT, VPN hoặc Bastion để kiểm soát truy cập.

Route table, Internet Gateway và NAT

Route table xác định gói tin đi theo hướng nào.

- Internet Gateway cho phép subnet công cộng truy cập Internet.
- NAT Gateway cho phép tài nguyên trong subnet riêng đi ra Internet nhưng không nhận kết nối trực tiếp từ ngoài vào.
- Định tuyến sai có thể làm hệ thống không truy cập được hoặc bị lộ ra Internet.



Security Group và Network ACL

Tiêu chí	Security Group	Network ACL
Phạm vi	Mức máy ảo/giao diện mạng	Mức subnet
Trạng thái	Stateful	Thường stateless
Quy tắc	Cho phép traffic cần thiết	Cho phép hoặc chặn theo rule
Sử dụng	Kiểm soát truy cập dịch vụ	Lớp lọc bổ sung cho subnet

Bảng quy tắc Security Group (Tường lửa) mẫu giữa các Subnet

Để đảm bảo quy tắc tối thiểu quyền truy cập (Principle of Least Privilege):

Nằm tại Subnet	Tên Security Group	Chiều (Inbound)	Nguồn (Source IP / SG)	Cổng (Port)	Ý nghĩa
Public	SG-LoadBalancer	Inbound	0.0.0.0/0	80, 443	Cho phép mọi người truy cập Web.
Private	SG-AppServer	Inbound	SG-LoadBalancer	8080 / 3000	**Chỉ** nhận kết nối từ Load Balancer.
Private	SG-Database	Inbound	SG-AppServer	3306 / 5432	**Tuyệt đối chỉ** nhận kết nối từ App Server.

Cân bằng tải trong IaaS

- Load balancer phân phối yêu cầu đến nhiều máy chủ backend.
- Giúp tăng hiệu năng, tính sẵn sàng và khả năng bảo trì.
- Có thể kiểm tra sức khỏe máy chủ và tự loại máy lỗi khỏi nhóm phục vụ.
- Thường kết hợp với autoscaling để đáp ứng tải biến động.

Tự động mở rộng tài nguyên

- Autoscaling tự động tăng hoặc giảm số lượng máy dựa trên chỉ số tải.
- Ví dụ: CPU vượt 70% trong 5 phút thì tăng thêm máy.
- Giúp đáp ứng tải cao nhưng vẫn tiết kiệm chi phí khi tải thấp.
- Cần cấu hình ngưỡng, số lượng tối thiểu/tối đa và thời gian ổn định.

Dịch vụ lưu trữ trong IaaS

- Block storage: gắn như ổ đĩa cho máy ảo.
- File storage: chia sẻ tệp qua giao thức mạng.
- Object storage: lưu đối tượng trong bucket/container.
- Snapshot và backup giúp phục hồi khi lỗi hệ thống hoặc lỗi người dùng.
- Chọn loại lưu trữ phải dựa trên hiệu năng, cách truy cập và chi phí.



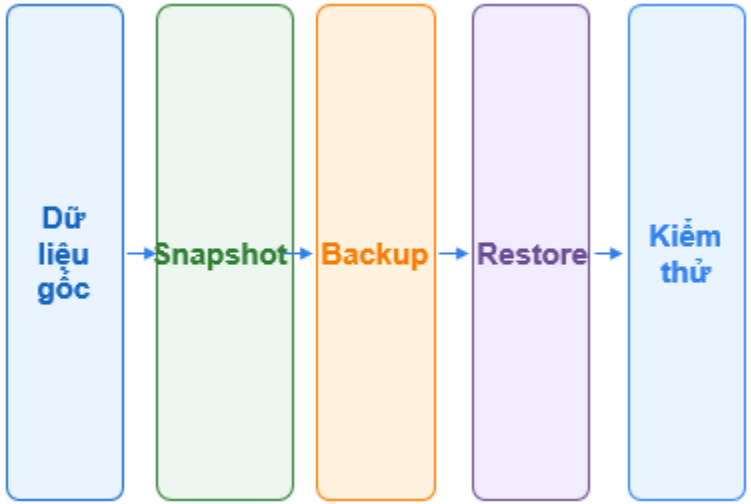
Block, File và Object Storage

- Không nên chọn lưu trữ chỉ theo giá; cần xét độ trễ, thông lượng, tính bền vững và cách ứng dụng truy cập dữ liệu.

Loại lưu trữ	Cách dùng	Phù hợp
Block	Gắn vào VM như đĩa	Hệ điều hành, database
File	Chia sẻ thư mục qua mạng	Dữ liệu dùng chung
Object	Lưu đối tượng qua API	Ảnh, video, backup, log

Snapshot, backup và khôi phục

- Snapshot ghi lại trạng thái đĩa tại thời điểm cụ thể.
- Backup là chiến lược bảo vệ dữ liệu có lịch, phạm vi và quy trình khôi phục rõ ràng.
- Cần kiểm thử khôi phục định kỳ để đảm bảo backup thực sự dùng được.
- Chính sách lưu giữ cần cân bằng giữa yêu cầu phục hồi và chi phí lưu trữ.



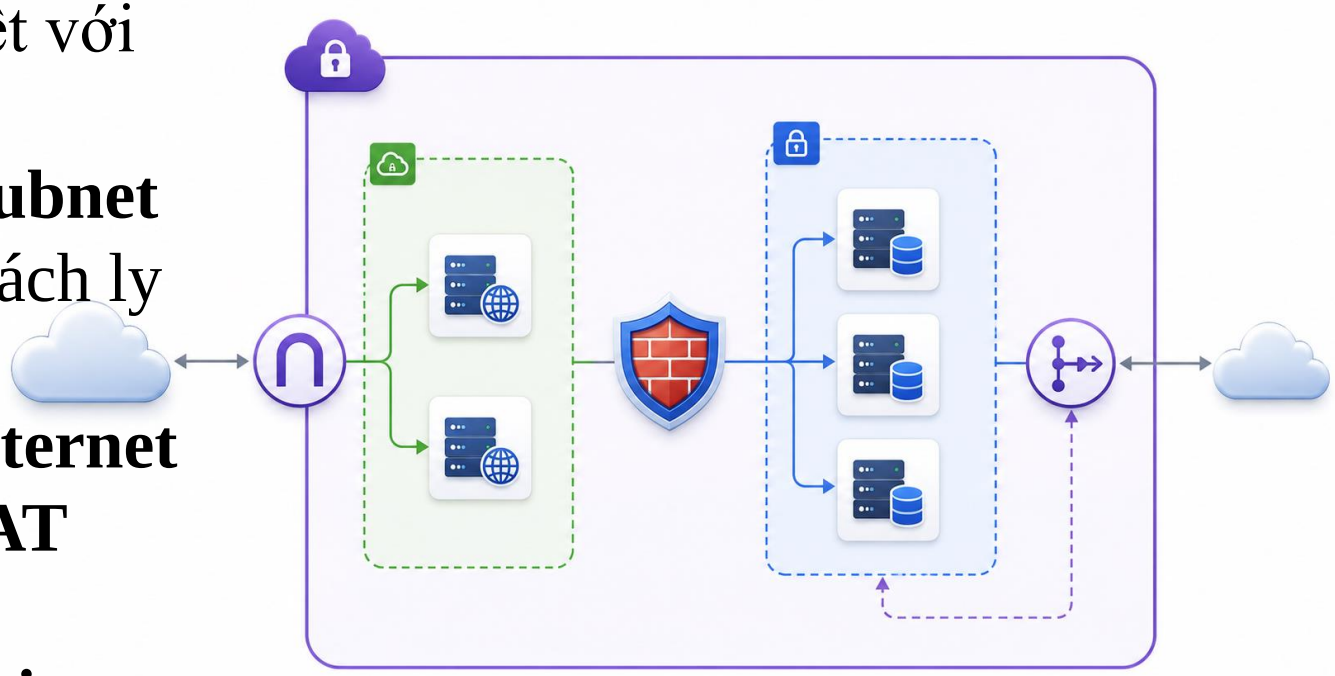
Quản lý tài nguyên IaaS

- Đặt tên tài nguyên có quy ước rõ ràng.
- Gắn nhãn theo môi trường, dự án, chủ sở hữu và trung tâm chi phí.
- Theo dõi quota, giới hạn dịch vụ và mức sử dụng thực tế.
- Dọn dẹp tài nguyên không còn sử dụng: IP, volume, snapshot, load balancer.
- Tự động hóa triển khai để giảm sai sót thủ công.



Bài thực hành 2.1: xây dựng mạng cloud cơ bản

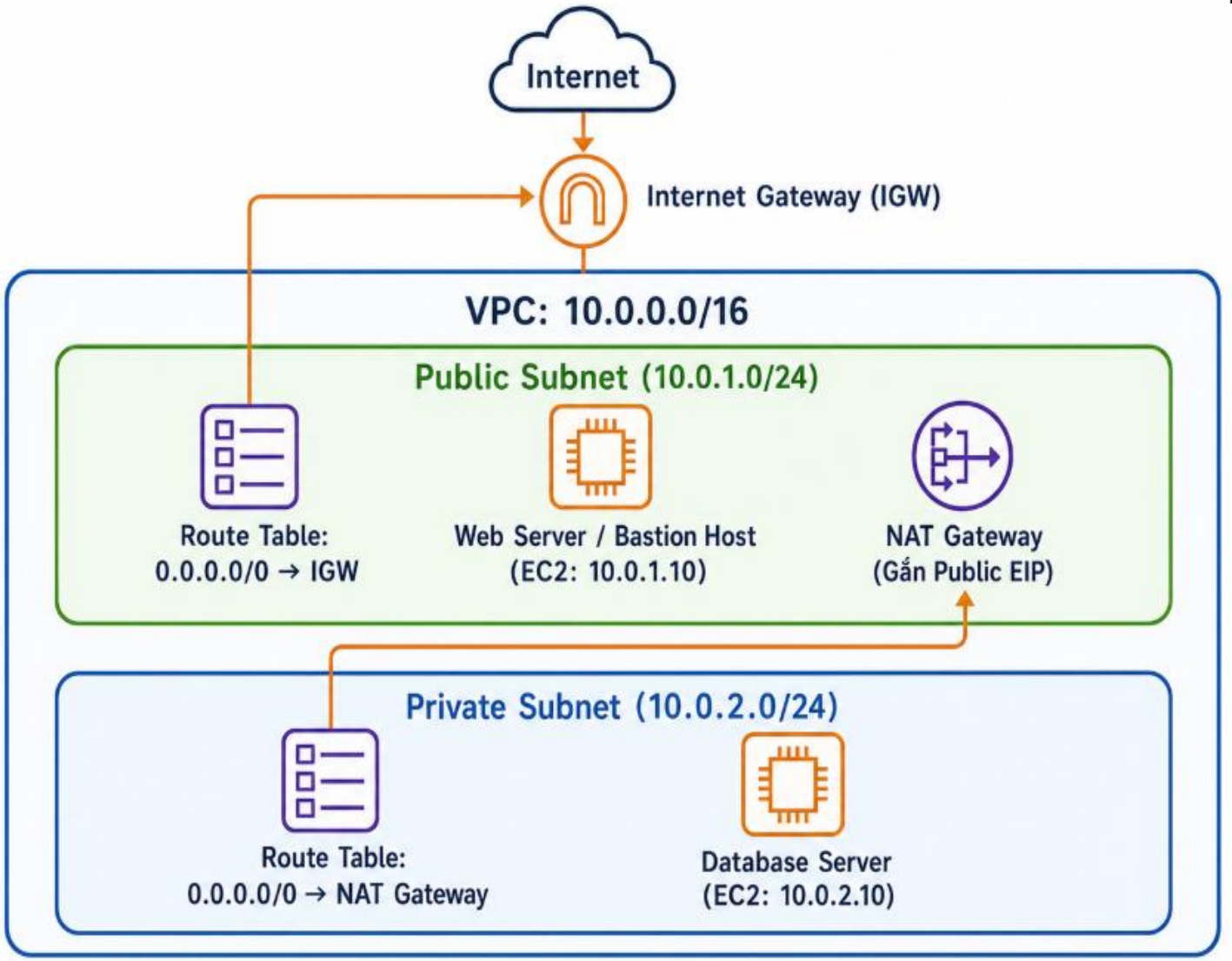
1.Khởi tạo một VPC isolated riêng biệt với dải IP riêng.
2.Cấu hình phân tầng mạng: **Public Subnet** (mở ra Internet) và **Private Subnet** (cách ly hoàn toàn).
3.Định tuyến lưu lượng mạng bằng **Internet Gateway (IGW)**, **Route Table**, và **NAT Gateway**.
4.Quản lý truy cập an toàn bằng **Security Group** và **Network ACL**.
5.Triển khai và kiểm tra kết nối từ Web Server đến Database Server.



Sơ đồ kiến trúc mạng đám mây hiện đại

Sơ đồ kiến trúc (Network Topology)

- Bài lab này được thiết kế theo mô hình **Mạng Cloud 2 lớp (2-Tier Architecture)** điển hình trên **AWS** (có thể áp dụng tương tự trên Azure hay GCP). Bài thực hành bao gồm: một phân vùng công khai (**Public Subnet**) chứa Web Server/Bastion Host và một phân vùng nội bộ (**Private Subnet**) chứa Database, đảm bảo tiêu chuẩn an toàn thông tin cơ bản.

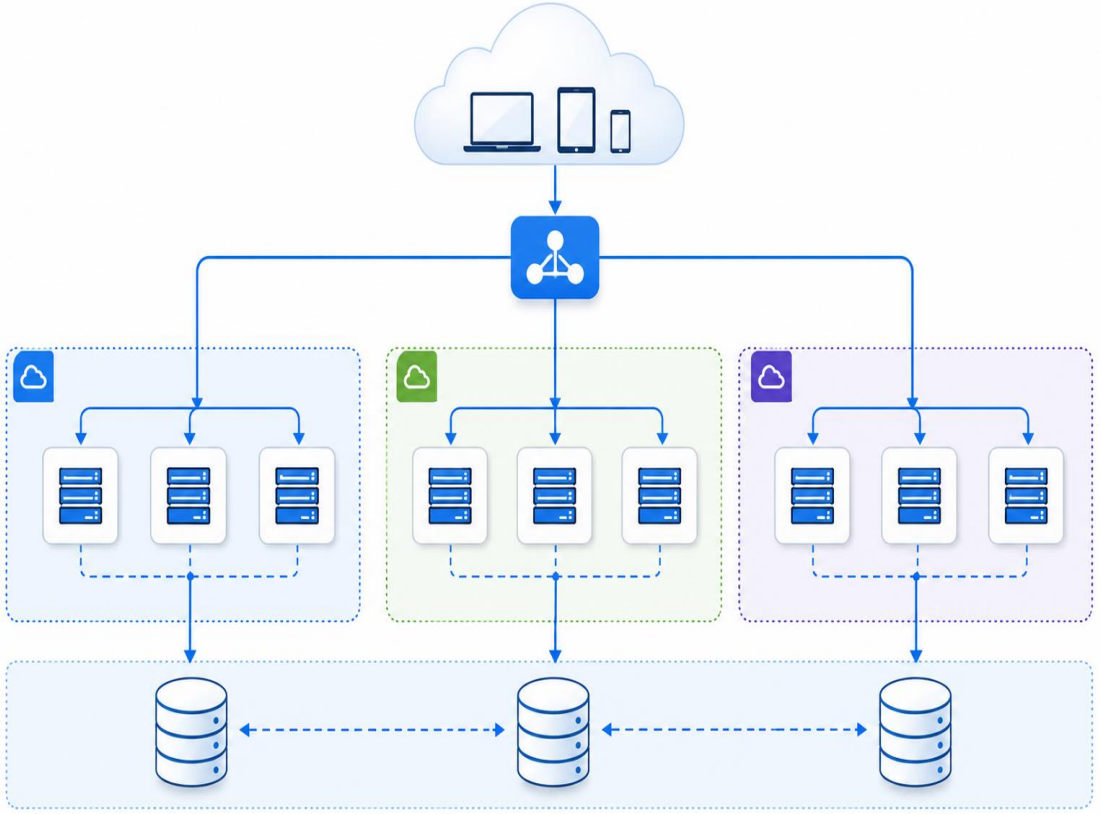


Bảng tổng hợp các thông số cấu hình

Thành phần	Tên gợi ý	Dải IP / Đích (Target)	Chức năng chính
VPC	Lab-VPC-Core	10.0.0.0/16	Không gian mạng cô lập toàn cục
Public Subnet	Public-Subnet-1	10.0.1.0/24	Chứa tài nguyên giao tiếp ra Internet
Private Subnet	Private-Subnet-1	10.0.2.0/24	Chứa tài nguyên nội bộ/Database
Internet Gateway	Lab-IGW	0.0.0.0/0 -> IGW	Cầu nối Bidirectional ra Internet
NAT Gateway	Lab-NAT-GW	0.0.0.0/0 -> NAT-GW	Cho phép Private Subnet đi ra ngoài (Unidirectional)

Tình huống IaaS: kiến trúc Web 3 lớp

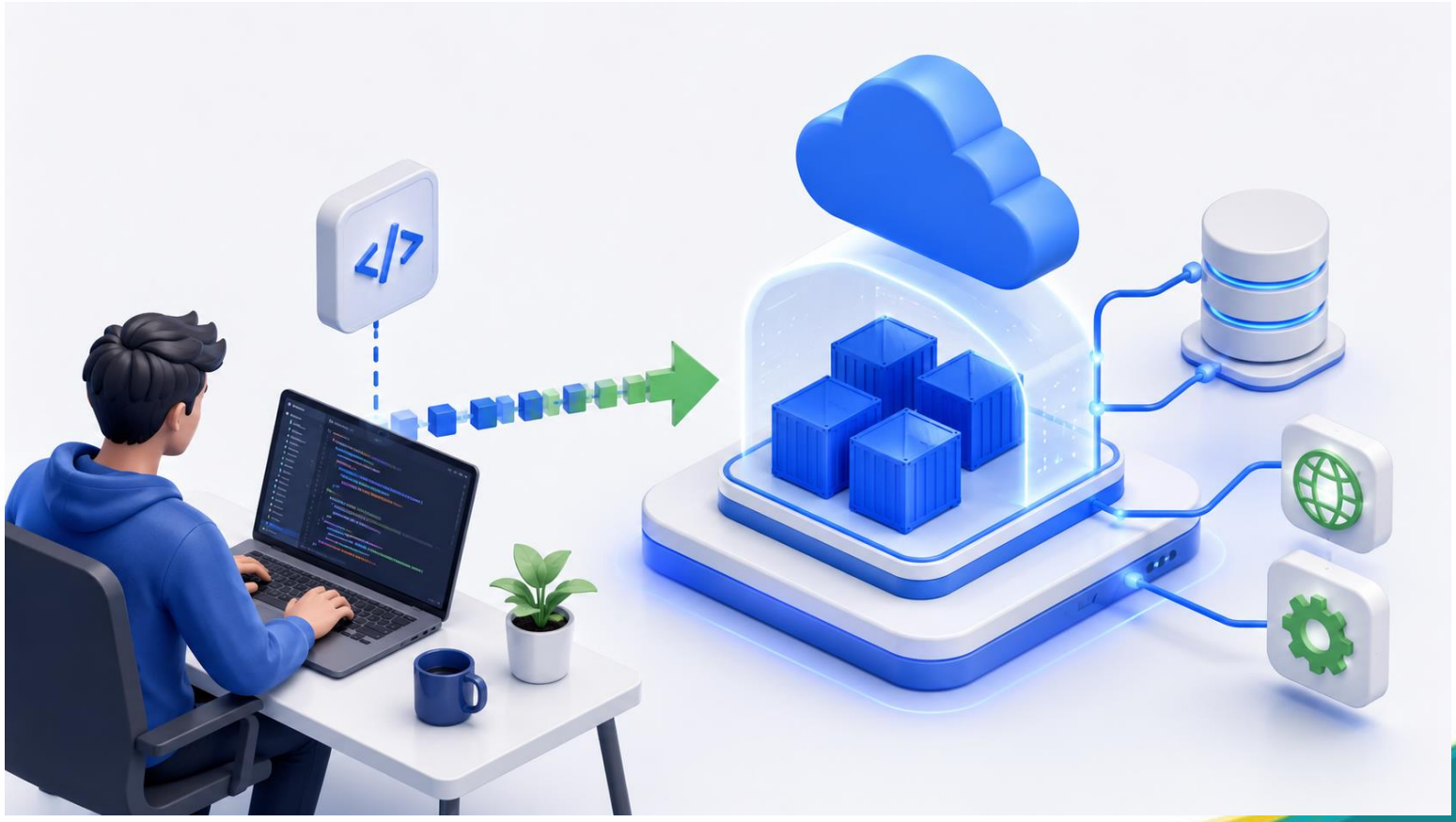
- Lớp Web đặt sau load balancer.
- Lớp ứng dụng đặt trong subnet riêng.
- Lớp database không mở truy cập trực tiếp từ Internet.
- Autoscaling áp dụng cho Web/App tùy tải.
- Backup, monitoring và security group là thành phần bắt buộc.



Sơ đồ kiến trúc mạng đám mây

Dịch vụ nền tảng - PaaS

- PaaS giúp lập trình viên tập trung vào mã nguồn và dữ liệu thay vì quản trị máy chủ

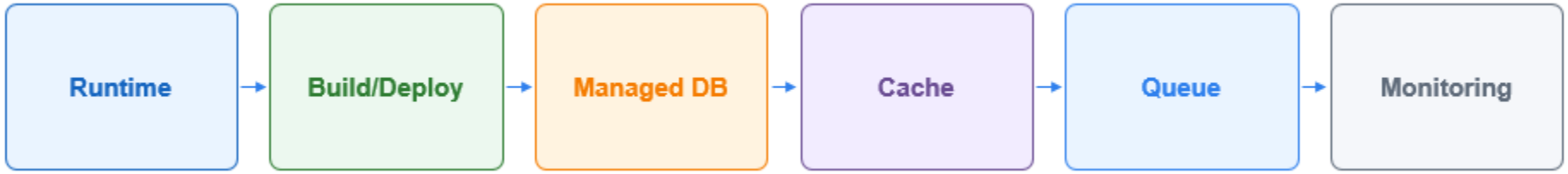


Lập trình viên triển khai mã lên nền tảng đám mây.

PaaS là gì?

- PaaS (Platform as a Service) cung cấp môi trường chạy ứng dụng được quản lý.
- Nhà cung cấp quản lý hệ điều hành, runtime, middleware và một phần cơ chế mở rộng.
- Nhà phát triển tập trung vào mã nguồn, cấu hình ứng dụng và dữ liệu.
- PaaS phù hợp khi cần triển khai nhanh và giảm gánh nặng vận hành máy chủ.

Các thành phần thường có trong PaaS



- Runtime: Java, Node.js, Python, .NET hoặc môi trường tương đương.
- Dịch vụ dữ liệu: database, cache, object storage, message queue.
- Vận hành: autoscaling, log, metric, health check.
- Bảo mật: biến môi trường, secret, TLS, IAM tích hợp.

Môi trường thực thi (Runtime Environment / Application Execution Framework)

- **Môi trường ngôn ngữ (Language Runtimes):** Hỗ trợ sẵn các trình biên dịch/thông dịch cho các ngôn ngữ phổ biến như Node.js, Python, Java, Go, PHP, .NET, Ruby,...
- **Web Server & Application Server:** Dựng sẵn các máy chủ web/ứng dụng như Nginx, Apache, Tomcat, Puma để xử lý HTTP/HTTPS requests.
- **Container Engine & Orchestration:** Các nền tảng PaaS hiện đại thường tích hợp sẵn Docker và Kubernetes bên dưới để đóng gói và điều phối ứng dụng tự động.

Công cụ phát triển & Triển khai (Development & Deployment Tools)

- **Tích hợp CI/CD (Continuous Integration / Continuous Deployment):** Tự động hóa quy trình Build, Test và Deploy khi lập trình viên push code (tích hợp với Git, GitHub, GitLab, Bitbucket).
- **Bộ công cụ dòng lệnh (CLI) & SDKs:** Cho phép quản lý ứng dụng, cấu hình tài nguyên trực tiếp từ terminal của nhà phát triển.
- **Môi trường lập trình (Web-based IDE / Integration):** Hỗ trợ viết code, debug trực tiếp trên giao diện web hoặc kết nối mượt mà với VS Code, IntelliJ,...

Dịch vụ dữ liệu & Lưu trữ (Database & Data Management Services)

PaaS cung cấp các hệ quản trị cơ sở dữ liệu dưới dạng dịch vụ (DBaaS - Database as a Service), giúp quản lý dữ liệu mà không cần tự cài đặt DB Server.

- **Relational Database (SQL):** PostgreSQL, MySQL, MariaDB, SQL Server.
- **NoSQL Database:** MongoDB, Redis (dùng cho Caching/In-memory store), Cassandra.
- **Dịch vụ lưu trữ tệp (Object Storage / Block Storage):** Tích hợp dịch vụ lưu trữ file tĩnh, hình ảnh, tài liệu (như AWS S3, Google Cloud Storage).

Dịch vụ tích hợp & Middleware (Integration Services & Middleware)

Cung cấp các thành phần trung gian kết nối các microservices hoặc hệ thống bên ngoài.

- **Message Broker / Queue:** RabbitMQ, Apache Kafka, AWS SQS để xử lý giao tiếp bất đồng bộ giữa các dịch vụ.
- **API Gateway & Management:** Quản lý, định tuyến API, cân bằng tải API và giới hạn lưu lượng (rate limiting).
- **Dịch vụ định danh & Xác thực (Identity & Authentication):** Hỗ trợ OAuth2, OIDC, JWT, SSO để quản lý người dùng nhanh chóng.

Thành phần Quản trị, Giám sát & Co giãn (Management, Monitoring & Auto-scaling)

Giúp duy trì tính sẵn sàng và hiệu năng của ứng dụng.

- **Tự động mở rộng (Auto-scaling):** Tự động tăng/giảm số lượng instance hoặc tài nguyên (CPU/RAM) dựa trên lưu lượng truy cập thực tế.
- **Giám sát & Ghi log (Logging, Monitoring & APM):**
  - *Metrics:* Theo dõi dung lượng RAM, CPU, thời gian phản hồi (Response time).
  - *Logging:* Tập trung log hệ thống và log ứng dụng (ELK Stack, Datadog).
  - *Health Check:* Phát hiện lỗi và tự động khởi động lại các container/instance bị crash.
- **Cân bằng tải (Load Balancer):** Phân phối lượng truy cập đồng đều tới các phiên bản (instances) đang chạy của ứng dụng.

Lớp Bảo mật & Tuân thủ (Security & Governance)

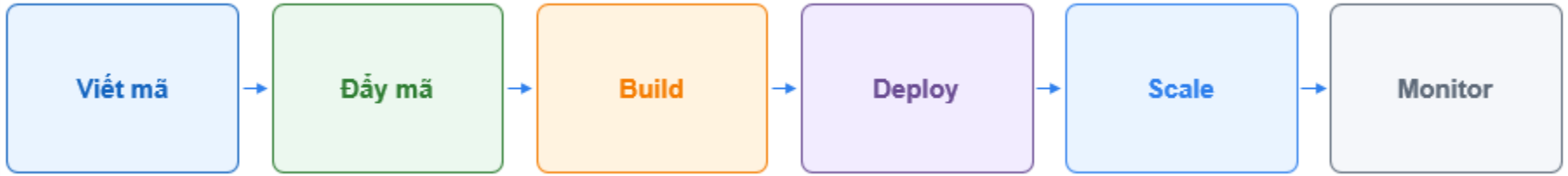
Đảm bảo an toàn cho ứng dụng và dữ liệu trên nền tảng PaaS.

- **Quản lý chứng chỉ SSL/TLS:** Tự động cấp phát và gia hạn chứng chỉ HTTPS (như Let's Encrypt).
- **IAM (Identity & Access Management):** Phân quyền truy cập chi tiết cho các thành viên trong nhóm phát triển.
- **Bảo mật mạng (Network Isolation / VPC):** Phân chia mạng nội bộ, thiết lập Firewall, chống tấn công DDoS.
- **Quản lý bí mật (Secrets Management):** Mã hóa và lưu trữ an toàn các biến môi trường (Environment Variables), API Keys, Database Password.

Ví dụ về các sản phẩm PaaS phổ biến thực tế:

- **Public PaaS nổi tiếng:** Heroku, Google App Engine, AWS Elastic Beanstalk, Azure App Service, Vercel, Render.
- **Enterprise / Container PaaS:** Red Hat OpenShift, Cloud Foundry.

Luồng triển khai ứng dụng trên PaaS



- PaaS thường tích hợp pipeline/flow đơn giản từ mã nguồn đến môi trường chạy.
- Có thể triển khai từ Git, file đóng gói hoặc container.
- Nền tảng tự quản lý health check, log và một phần cân bằng tải.
- Người dùng vẫn cần cấu hình biến môi trường, secret và kết nối database.

Luồng triển khai ứng dụng trên PaaS

- Khi triển khai một ứng dụng trên nền tảng **PaaS (Platform as a Service)**, quy trình (pipeline/flow) đã được tự động hóa tối đa. Nhà phát triển chỉ cần tập trung vào khâu viết mã nguồn (*code*) và cấu hình ứng dụng, toàn bộ khâu chuẩn bị môi trường, biên dịch, đóng gói, phân phối và mở rộng sẽ do hệ thống PaaS đảm nhận.
- **Luồng triển khai ứng dụng trên PaaS chuẩn 6 bước** từ môi trường lập trình local cho đến khi ứng dụng chạy thực tế (*Production*):

Bước 1: Chuẩn bị mã nguồn & Cấu hình (Development & Configuration)

Ở máy lập trình local, nhà phát triển chuẩn bị mã nguồn và thiết lập các tệp cấu hình cần thiết để PaaS hiểu cách vận hành ứng dụng:

- **Khai báo phụ thuộc (Dependencies):** Định nghĩa danh sách thư viện trong các file tiêu chuẩn (ví dụ: package.json cho Node.js, requirements.txt cho Python, pom.xml cho Java).
- **File cấu hình PaaS (Manifest/Procfile):** Cung cấp các lệnh khởi chạy ứng dụng (ví dụ: Procfile trên Heroku, app.yaml trên Google App Engine, render.yaml trên Render).
- **Quản lý biến môi trường (Environment Variables):** Đưa các thông tin nhạy cảm (API Key, Database URL, Secret Key) vào biến môi trường trên PaaS thay vì lưu cứng trong code.

Bước 2: Đẩy mã nguồn lên hệ thống (Code Push / Trigger)

Lập trình viên chuyển giao mã nguồn lên PaaS bằng một trong hai cách phổ biến:

- **Qua Git (Khuyên dùng/Phổ biến nhất):** Thực hiện git push về kho lưu trữ (GitHub/GitLab) được liên kết với PaaS, hoặc push trực tiếp vào Git remote của PaaS (ví dụ: git push heroku main).
- **Qua CLI / Web Dashboard:** Sử dụng bộ công cụ dòng lệnh (như gcloud app deploy, aws eb deploy) hoặc nén file mã nguồn/container image và tải trực tiếp lên giao diện Web.

Bước 3: Tự động Biên dịch & Đóng gói (Build Phase)

Khi nhận được mã nguồn mới, Build Engine của PaaS sẽ tự động thực hiện các tác vụ:

- **Phát hiện ngôn ngữ (Language Detection):** Phân tích mã nguồn để nhận biết công nghệ (Node.js, Python, Go,...).
- **Tải & Cài đặt thư viện:** Tự động chạy các lệnh cài đặt phụ thuộc (như npm install, pip install).
- **Biên dịch (Compile):** Biên dịch mã nguồn (nếu là các ngôn ngữ biên dịch như Java, Go, C#) hoặc build các tài nguyên tĩnh (Frontend assets like React/Vue).
- **Tạo Image/Slug**: Đóng gói toàn bộ ứng dụng cùng môi trường runtime thành một gói ứng dụng hoàn chỉnh (thường là Container Image hoặc Slug) sẵn sàng để chạy.

Bước 4: Triển khai & Khởi chạy (Deployment & Execution)

Sau khi đóng gói thành công, PaaS tiến hành đưa ứng dụng vào hoạt động:

- **Cấp phát container/instance:** Khởi tạo các container ảo (như Worker/Dyno/Pod) dựa trên thông số cấu hình.
- **Inject biến môi trường:** Tải các biến môi trường và thông số kết nối Database/Redis vào container.
- **Khởi chạy ứng dụng:** Thực thi lệnh startup (đã khai báo ở Bước 1).
- **Kiểm tra sức khỏe (Health Check):** PaaS gửi yêu cầu kiểm tra xem ứng dụng đã sẵn sàng nhận kết nối hay chưa trước khi cắt lưu lượng truy cập sang phiên bản mới.

Bước 5: Định tuyến & Cấp phát Tên miền/SSL (Routing & Ingress)

- **Cập nhật Router / Load Balancer:** Hệ thống Load Balancer của PaaS cập nhật bảng định tuyến, bắt đầu điều hướng lưu lượng truy cập (Traffic) người dùng vào phiên bản ứng dụng mới.
- **Chiến lược triển khai an toàn (Zero-downtime):**PaaS thường áp dụng Rolling Update hoặc Blue-Green Deployment (chỉ ngắt phiên bản cũ sau khi phiên bản mới đã chạy ổn định 100%), giúp người dùng không bị gián đoạn dịch vụ.
- **Tự động gắn SSL/TLS:** Tự động cấp phát chứng chỉ HTTPS và định tuyến tên miền (Domain) tùy chỉnh.

Bước 6: Giám sát, Bảo trì & Co giãn (Monitoring & Auto-scaling)

Khi ứng dụng đã chạy thực tế, PaaS tiếp tục vận hành tự động theo chu kỳ:

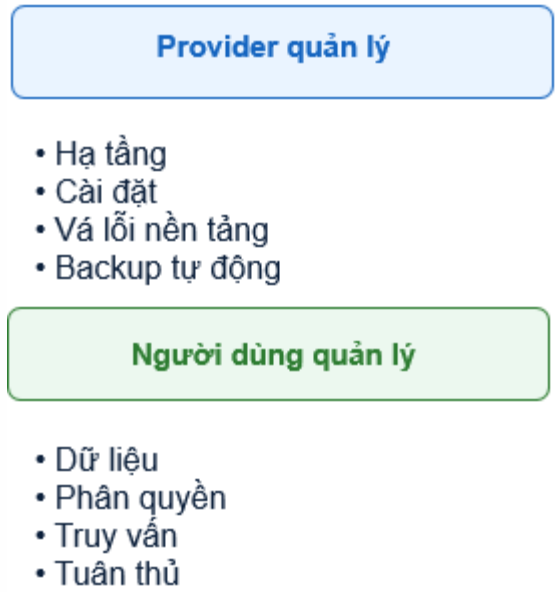
- **Thu thập Log & Metric:** Gom tập trung toàn bộ log của ứng dụng và theo dõi các chỉ số CPU, RAM, Latency, Error Rate.
- **Tự động khôi phục (Self-healing):** Nếu một instance ứng dụng bị crash hoặc đơ, PaaS sẽ tự động kill và khởi tạo lại instance mới.
- **Tự động co giãn (Auto-scaling):** Nếu lượng truy cập tăng đột biến, PaaS tự động tăng thêm số lượng instance (Horizontal Scaling) hoặc nâng RAM/CPU (Vertical Scaling) và ngược lại khi lưu lượng giảm.

Luồng triển khai PaaS

- **Tóm lại:** Luồng triển khai PaaS giúp rút ngắn quy trình chuyển giao từ Code → Production chỉ còn trong vài phút. Lập trình viên không cần can thiệp vào các thao tác thủ công như SSH vào máy chủ, cài đặt Nginx, cấu hình Systemd hay gia hạn SSL.

Dịch vụ cơ sở dữ liệu được quản lý

- Managed Database giúp giảm công việc cài đặt, vá lỗi và backup thủ công.
- Các tính năng thường gặp: backup tự động, bản sao đọc, mở rộng dung lượng, giám sát.
- Người dùng vẫn phải thiết kế lược đồ cơ sở dữ liệu (schema), phân quyền, tối ưu truy vấn và bảo vệ dữ liệu.
- Không nên coi managed service là “không cần quản trị”.



Ưu điểm và hạn chế của PaaS

Tiêu chí	Ưu điểm	Hạn chế
Tốc độ triển khai	Rất nhanh (vài phút)	—
Quản lý hạ tầng	Tự động hoàn toàn	Mất quyền kiểm soát sâu tầng bên dưới
Chi phí ban đầu	Thấp, không cần mua phần cứng	—
Chi phí khi Scale lớn	—	Đắt hơn so với IaaS/On-premise
Tính linh hoạt	Tập trung vào code	Bị giới hạn bởi môi trường PaaS quy định
Khả năng di chuyển	—	Dễ bị Vendor Lock-in

Ưu điểm và hạn chế của PaaS

- **Tối ưu thời gian đưa sản phẩm ra thị trường (Time-to-Market)**
- **Loại bỏ khâu setup hạ tầng:** Nhà phát triển không cần tốn thời gian mua sắm, cài đặt, cấu hình hệ điều hành, máy chủ web hay hệ quản trị cơ sở dữ liệu.
- **Tập trung 100% vào Business Logic:** Đội ngũ phát triển chỉ cần tập trung viết code cho tính năng sản phẩm, giúp rút ngắn chu kỳ phát triển và triển khai tính năng mới từ vài tuần xuống còn vài giờ.

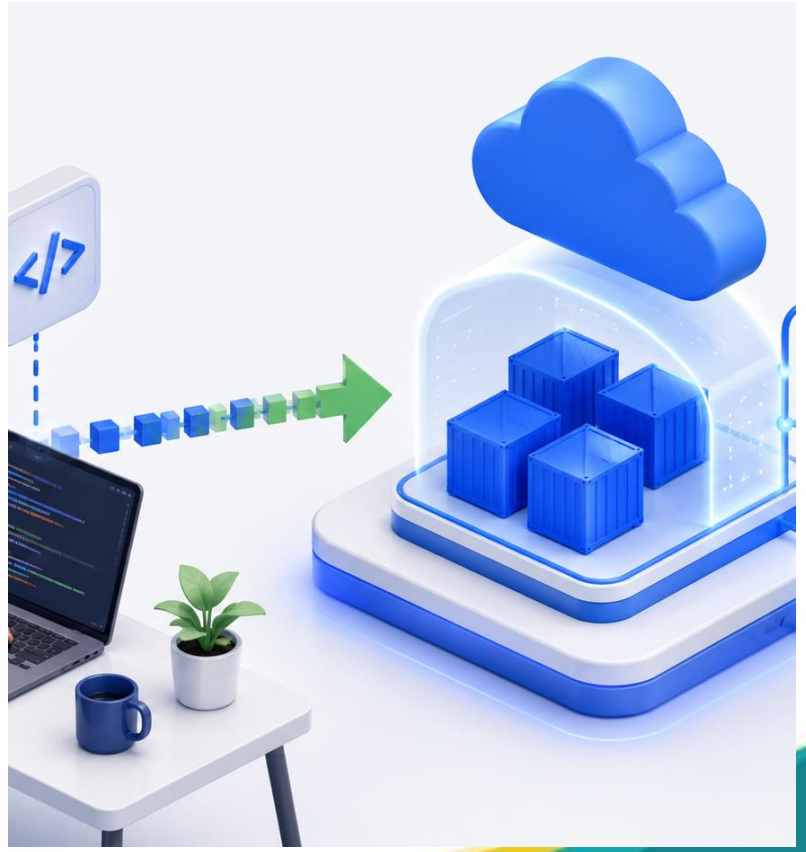
Ưu điểm và hạn chế của PaaS

Hạn chế của PaaS:

- Khóa chặt vào nhà cung cấp (Vendor Lock-in)
- Giới hạn khả năng tùy biến (Lack of Control & Customization)
- Rủi ro về Bảo mật & Tuân thủ (Security & Compliance Issues)
- Chi phí tăng nhanh khi mở rộng quy mô lớn (Cost Inflation at Scale)

Khi nào nên chọn PaaS?

- **Khuyên dùng PaaS khi:** Xây dựng MVP (sản phẩm khả thi tối thiểu), ứng dụng Web/Mobile/API cần ra mắt nhanh, dự án có quy mô nhỏ đến vừa, hoặc khi doanh nghiệp muốn tập trung tối đa nguồn lực vào việc phát triển tính năng thay vì vận hành hệ thống.
- Ứng dụng có kiến trúc tương đối chuẩn, ít cần tùy biến hệ điều hành.
- Cần tích hợp nhanh với database, cache, queue hoặc dịch vụ xác thực.
- Không phù hợp nếu ứng dụng cần kiểm soát kernel, driver hoặc cấu hình mạng rất đặc thù.



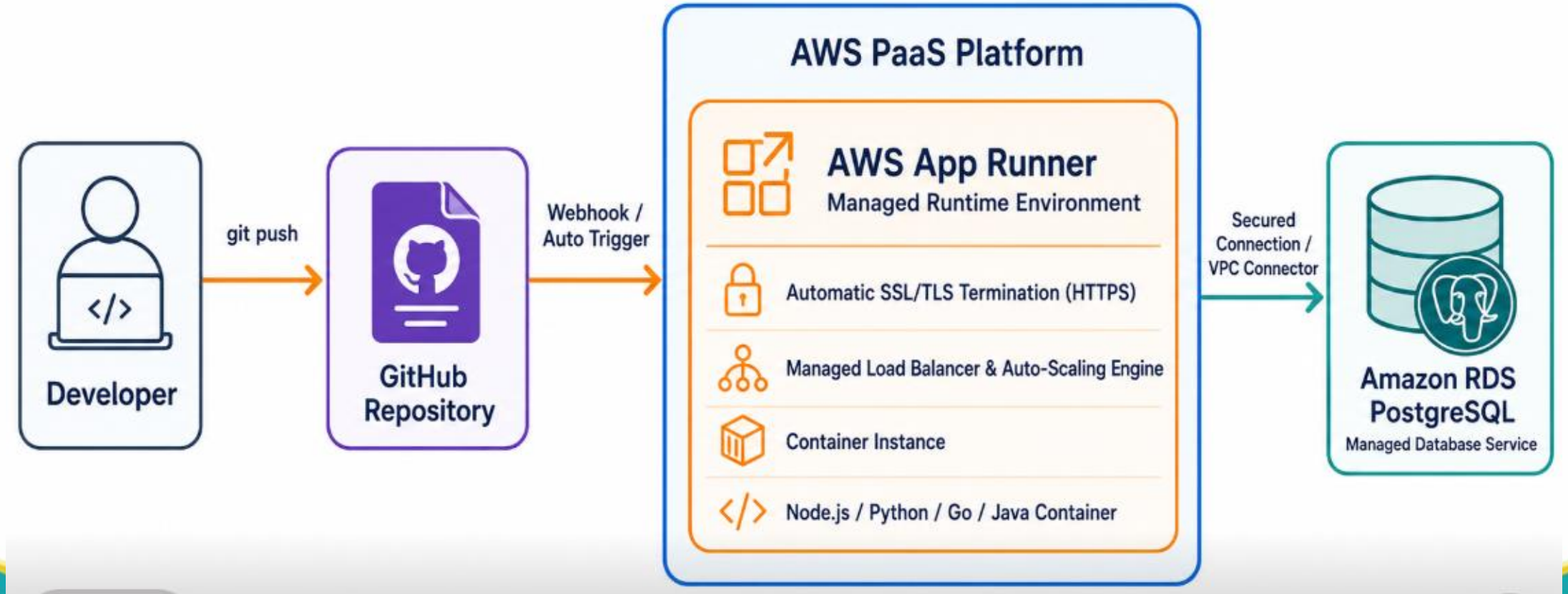
Bài thực hành 2.2: triển khai ứng dụng PaaS

- Khác với IaaS (nơi chúng ta phải tự quản lý VPC, Subnet, OS, Patching, Load Balancer như bài trước), **PaaS tối ưu hóa năng suất phát triển bằng cách trừu tượng hóa toàn bộ hạ tầng bên dưới**. Đội ngũ kỹ sư chỉ cần bàn giao mã nguồn (Source Code) hoặc Container Image, nền tảng PaaS sẽ tự động đảm nhận việc đóng gói, cấp phát tài nguyên, định tuyến, bảo mật và mở rộng quy mô (Auto-scaling).
- Trong bài lab này, chúng ta sẽ sử dụng **AWS App Runner** (hoặc áp dụng tư duy tương tự trên **Heroku**, **Render**, **Azure App Service**, **GCP Cloud Run**) để triển khai một ứng dụng Web Full-stack kết nối với Database managed service.

Mục tiêu bài thực hành

1. **Thấu hiểu tư duy PaaS:** Phân biệt rõ ranh giới trách nhiệm giữa người dùng (Code, Data) và Nhà cung cấp Cloud (Runtime, OS, Server, Network, Virtualization).
2. **Containerize ứng dụng**: Đóng gói ứng dụng web đơn giản bằng Dockerfile.
3. **Triển khai tự động (CI/CD):** Cấu hình tự động Build và Deploy khi có mã nguồn mới.
4. **Quản lý biến môi trường & Bảo mật**: Kết nối ứng dụng PaaS tới Database managed bảo mật.
5. **Kiểm thử Auto-scaling & Monitoring**: Quan sát khả năng tự động tăng/giảm tài nguyên dựa trên lượng truy cập.

Sơ đồ Mô hình Kiến trúc PaaS



So sánh Trải nghiệm Quản trị: IaaS vs. PaaS

Tiêu chí	Mô hình IaaS (Bài trước)	Mô hình PaaS (Bài này)
Quản trị Hạ tầng	Cấu hình thủ công VPC, Subnet, Route Table, IGW/NAT, Security Groups.	**Hoàn toàn ẩn.** Nhà cung cấp Cloud tự quản lý mạng và cụm Server.
Hệ điều hành & Runtime	Phải tự SSH, cài đặt Node.js, Nginx, vá lỗi OS (yum update).	**Managed.** Chọn phiên bản Runtime (Node/Python/Java), Cloud tự vá lỗi.
Load Balancer & SSL	Phải tự tạo ALB, mua/cấu hình AWS ACM Certificate.	**Tự động.** Có sẵn tên miền HTTPS miễn phí & Load Balancer tích hợp.
Tốc độ Triển khai	Mất từ 30 - 60 phút thiết lập ban đầu.	Mất từ 5 - 10 phút để đưa ứng dụng lên Internet.
Tính linh hoạt (Flexibility)	Rất cao (Có thể can thiệp sâu vào Kernel, Network, OS).	Bị giới hạn theo khuôn khổ và Runtime do nền tảng hỗ trợ.

Dịch vụ phần mềm - SaaS

- SaaS cung cấp phần mềm hoàn chỉnh, người dùng chủ yếu cấu hình và sử dụng.

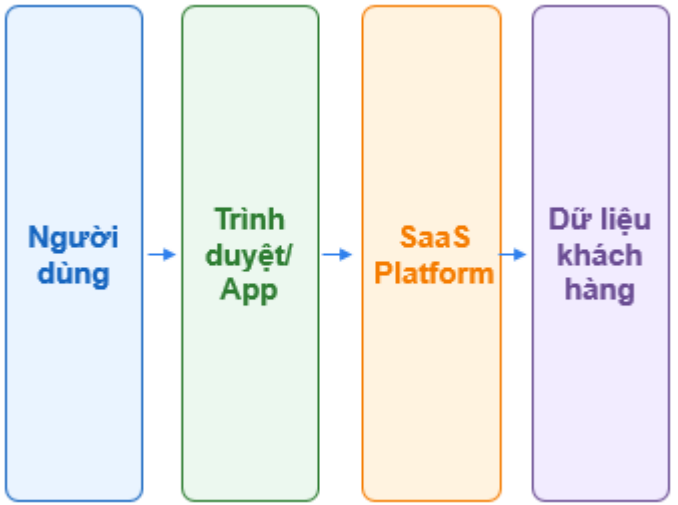


SaaS là gì?

- Software as a Service là phần mềm hoàn chỉnh được cung cấp qua Internet.
- Người dùng không quản trị máy chủ, hệ điều hành, runtime hoặc mã nguồn nền tảng.
- Hoạt động chủ yếu: đăng ký tài khoản, cấu hình tổ chức, phân quyền và sử dụng nghiệp vụ.
- Ví dụ: email doanh nghiệp, hệ thống CRM, LMS, quản lý nhân sự, văn phòng trực tuyến.

Đặc điểm của ứng dụng SaaS

- Truy cập qua trình duyệt hoặc ứng dụng khách.
- Cập nhật phần mềm do nhà cung cấp thực hiện.
- Thanh toán theo thuê bao, người dùng hoặc mức sử dụng.
- Có cơ chế quản lý tổ chức, nhóm, vai trò và dữ liệu khách hàng.
- Tích hợp với dịch vụ định danh và API của doanh nghiệp.



So sánh SaaS với Software Truyền thống (On-Premises)

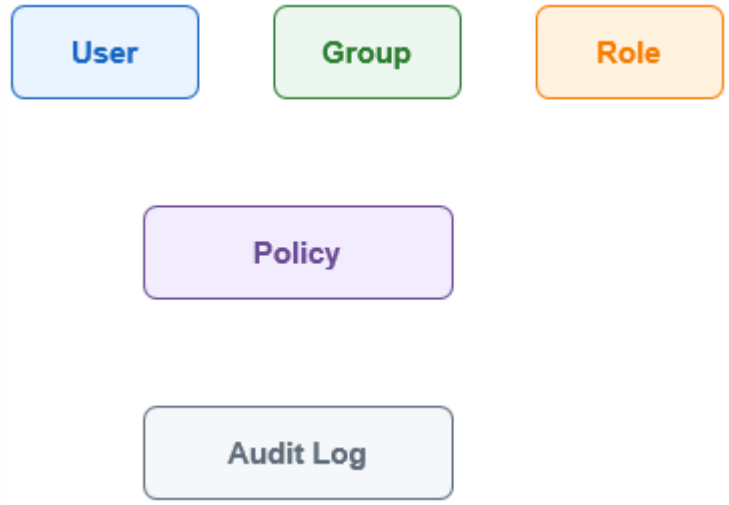
Tiêu chí	SaaS (Software as a Service)	On-Premises (Traditional Software)
Mô hình Đầu tư	**OpEx** (Chi phí hoạt động linh hoạt).	**CapEx** (Chi phí đầu tư phần cứng/license ban đầu rất lớn).
Thời gian Triển khai	**Ngay lập tức** (Chỉ cần đăng ký tài khoản).	Hàng tuần đến hàng tháng (Mua server, cài OS, setup DB, config network).
Quản trị & Bảo trì	Do nhà cung cấp SaaS đảm nhận 100%.	Đội ngũ IT nội bộ phải tự quản lý, vá lỗi, backup.
Cập nhật Tính năng	Tự động, liên tục, không tốn phí phụ trội.	Phải mua bản nâng cấp (Major Version) hoặc tốn công deploy.
An toàn Dữ liệu	Tuân thủ các tiêu chuẩn quốc tế (SOC2, ISO 27001, GDPR).	Phụ thuộc hoàn toàn vào năng lực an ninh mạng của đội ngũ IT nội bộ.

Kiến trúc đa thuê trong SaaS

- Nhiều khách hàng dùng chung một nền tảng phần mềm.
- Dữ liệu và cấu hình của từng khách hàng được tách biệt logic.
- Nhà cung cấp tối ưu vận hành nhờ dùng chung mã nguồn, hạ tầng và quy trình cập nhật.
- Cần thiết kế bảo mật dữ liệu, phân quyền và kiểm toán rất chặt.

Quản trị người dùng trong SaaS

- Tạo người dùng, nhóm và vai trò.
- Áp dụng xác thực đa yếu tố và chính sách mật khẩu.
- Tích hợp đăng nhập một lần nếu tổ chức có hệ thống định danh.
- Theo dõi nhật ký truy cập và hoạt động quản trị.
- Thu hồi tài khoản kịp thời khi nhân sự rời tổ chức.



Ưu điểm và hạn chế của SaaS

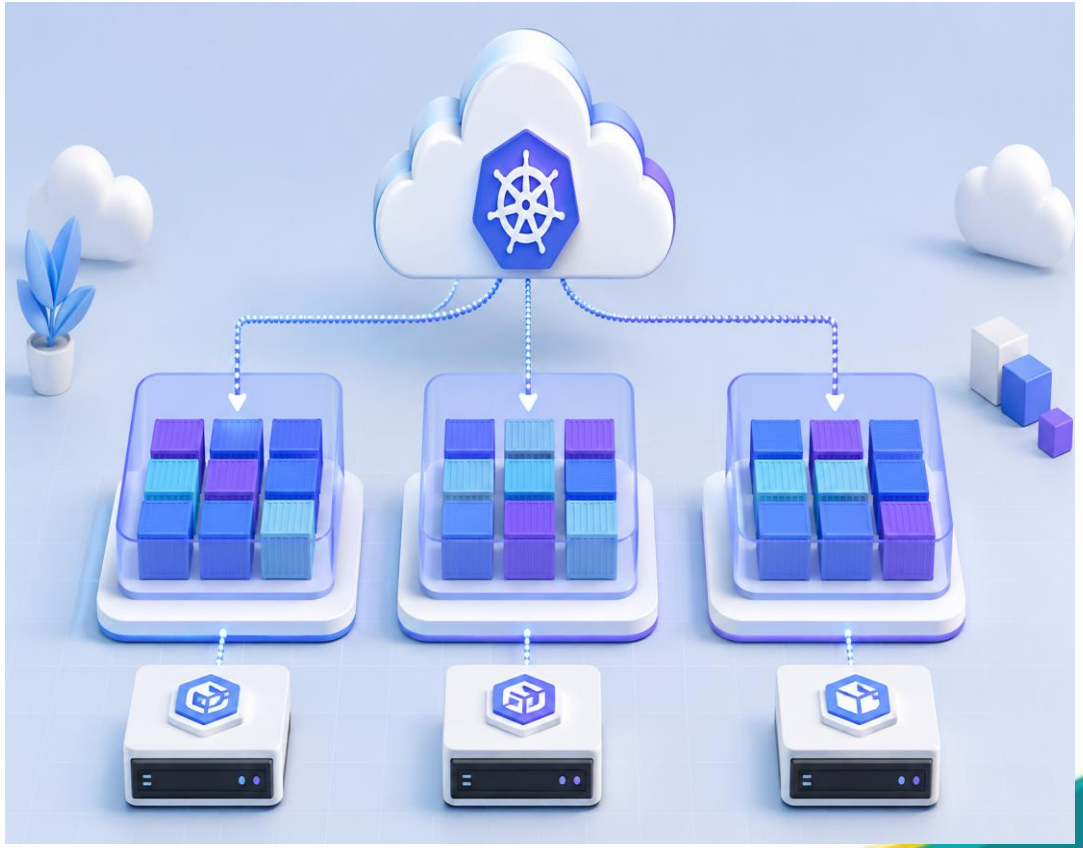
ƯU ĐIỂM	HẠN CHẾ
1.Đưa sản phẩm ra thị trường siêu nhanh 2.Tối ưu hóa chi phí (Chuyển CapEx->OpEx) 3.Tự động co giãn & Cập nhật liên tục 4.Truy cập linh hoạt từ mọi nơi 5.Tự động hóa công tác Bảo trì/Backup	1.Mất quyền kiểm soát Dữ liệu & Hạ tầng 2.Nguy cơ Khóa chặt dữ liệu (Vendor Lock) 3.Giới hạn khả năng Tuỳ biến (Custom) 4.Phụ thuộc hoàn toàn vào Kết nối Internet 5.Rủi ro Tích hợp với Hệ thống cũ (Legacy)

Khi nào nên chọn SaaS?

Tiêu chí	Nên chọn SaaS khi	NÊN CÂN NHẮC / KHÔNG chọn SaaS khi
Bản chất Nghiệp vụ	Các quy trình chuẩn hóa (HRM, CRM, Email, ERP chuẩn, Collaboration).	Các quy trình độc quyền, tạo nên lợi thế cạnh tranh cốt lõi của doanh nghiệp.
Yêu cầu An ninh	Cần tuân thủ các tiêu chuẩn chung (ISO 27001, SOC 2 Type II).	Dữ liệu thuộc bí mật quốc gia, y tế/tài chính có quy định ngặt nghèo về **Data Residency** (Bắt buộc lưu trữ On-Premises).
Năng lực IT	Đội ngũ IT mỏng, muốn tập trung phát triển sản phẩm chính.	Doanh nghiệp có đội ngũ System/DevOps mạnh, sở hữu Data Center riêng.
Ngân sách	Muốn phân bổ chi phí linh hoạt theo hàng tháng/năm.	Muốn đầu tư một lần dùng vĩnh viễn (Long-term CapEx optimization).

Container và CaaS

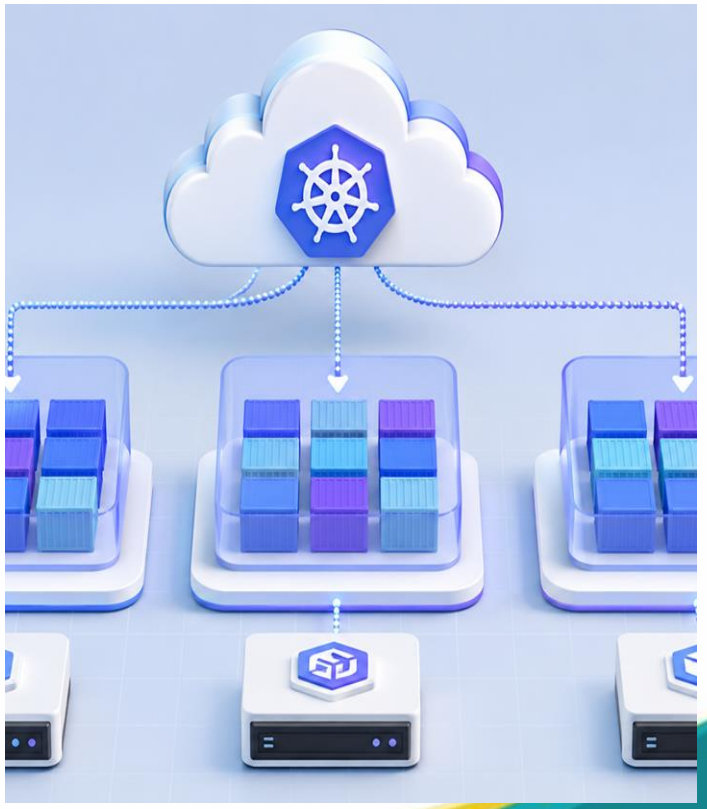
Container giúp đóng gói ứng dụng nhất quán và là nền tảng của nhiều kiến trúc cloud-native.



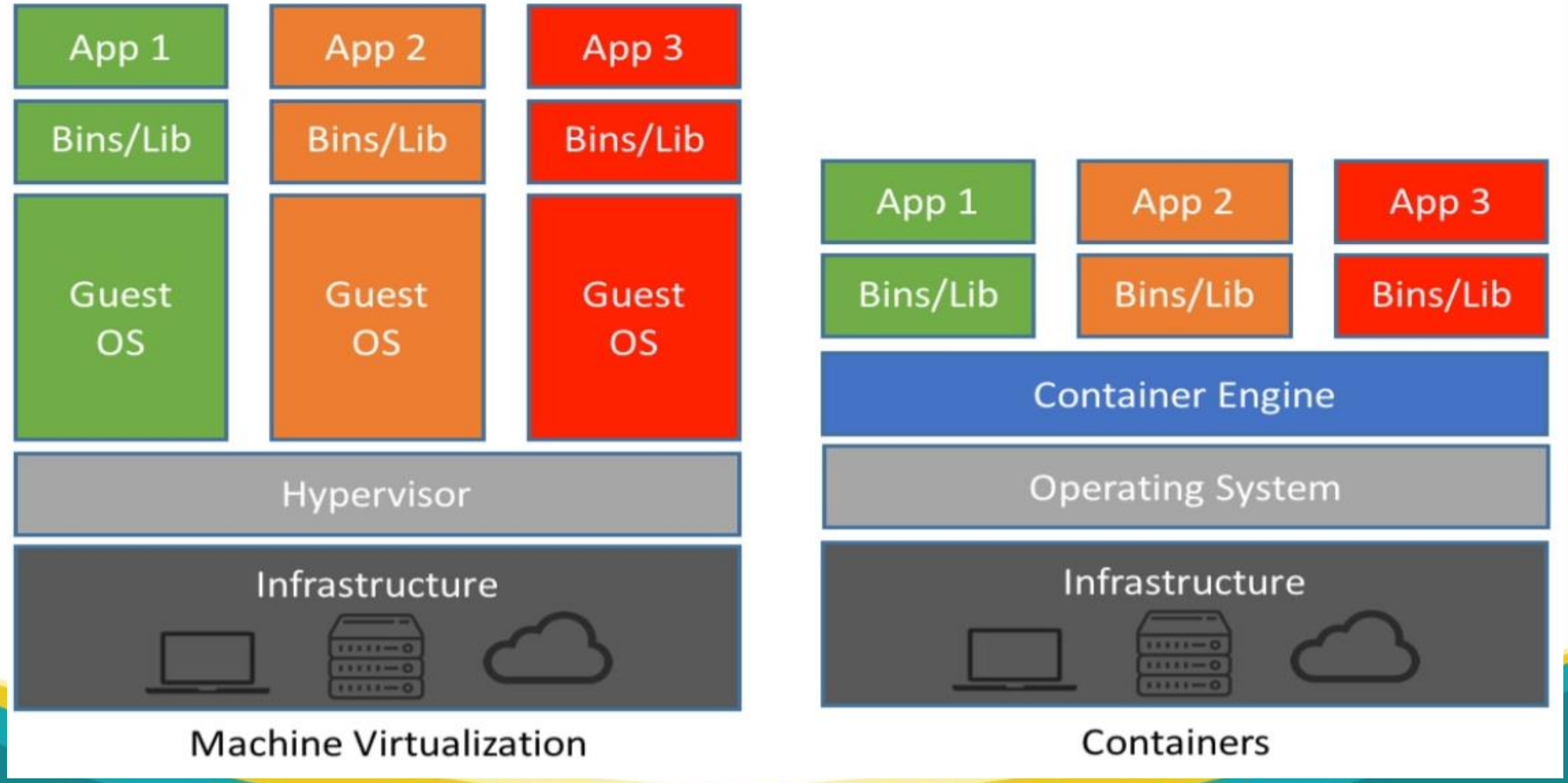
Điều phối đám mây và cụm container

Container là gì?

- Container đóng gói ứng dụng cùng thư viện và phụ thuộc cần thiết.
- Nhiều container dùng chung kernel của hệ điều hành host.
- Container khởi động nhanh, nhẹ hơn máy ảo và dễ di chuyển giữa môi trường.
- Container không phải là máy ảo thu nhỏ; cơ chế cô lập và vận hành khác VM.



Kiến trúc: Container vs. Virtual Machine (VM)



Kiến trúc: Container vs. Virtual Machine (VM)

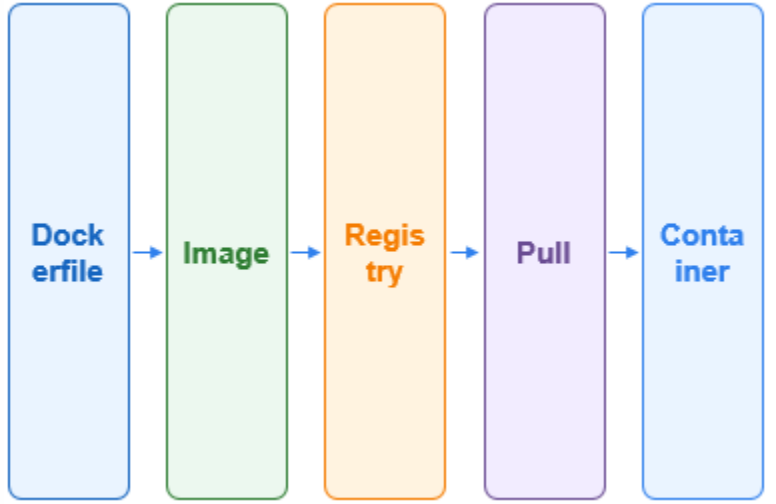
- Điểm khác biệt cốt lõi nằm ở chỗ **Container chia sẻ chung Kernel của Hệ điều hành Host (Host OS Kernel)**, trong khi Máy ảo (VM) phải chạy một Hệ điều hành khách riêng biệt (**Guest OS**).
- **Virtual Machine**: Mỗi ứng dụng nằm trên một Guest OS riêng, thông qua lớp Hypervisor. Điểm trừ là tốn dung lượng ổ đĩa (hàng triệu GB cho OS), ngốn RAM/CPU và khởi động lâu (vài phút).
- **Container:** Tất cả ứng dụng chạy chung trên một Container Engine (như Docker/containerd) và dùng chung Host OS. Nhờ lược bỏ được Guest OS, Container cực kỳ nhẹ (chỉ vài MB đến vài trăm MB) và khởi động gần như ngay lập tức (vài miligiây).

So sánh Container vs. Virtual Machine (VM)

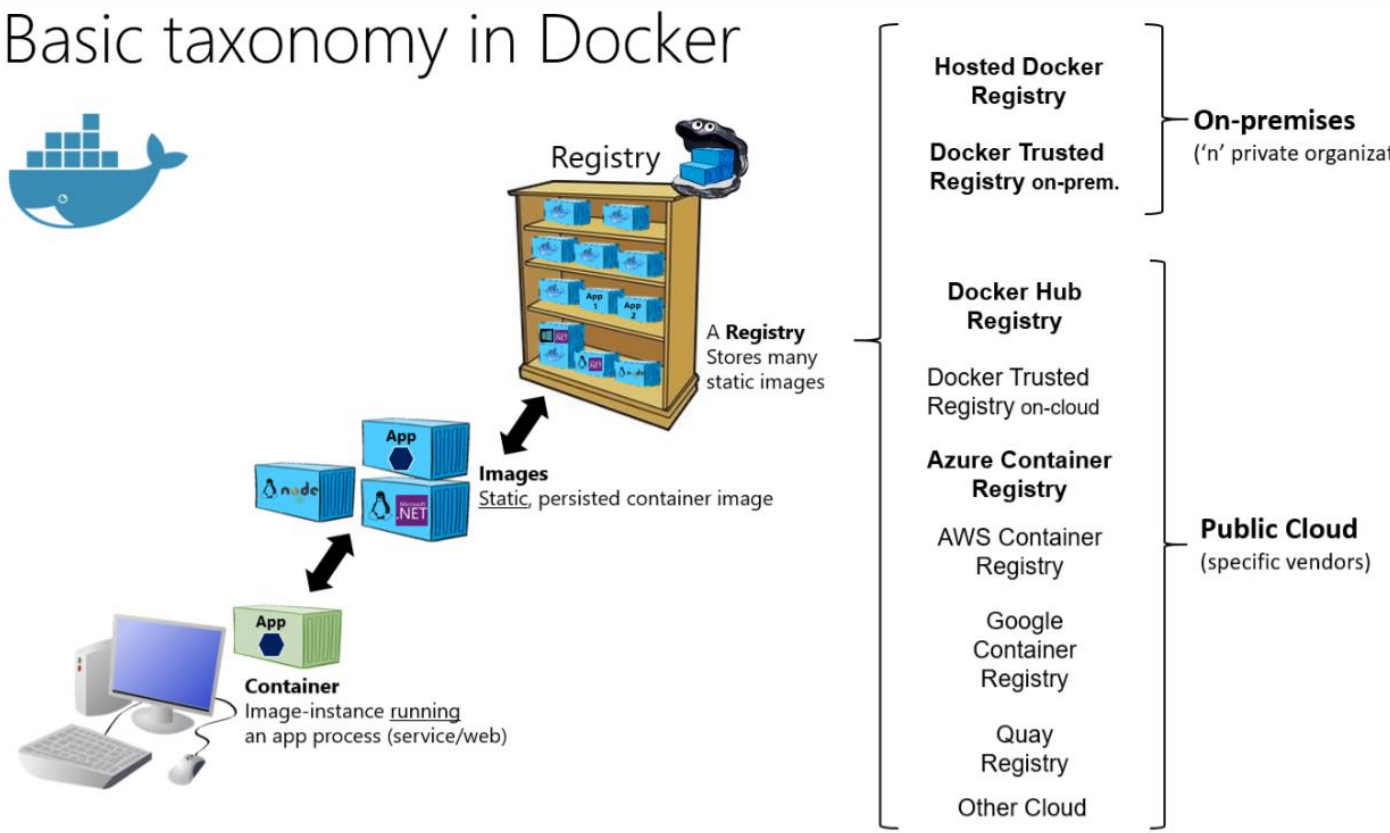
Tiêu chí	Container	Virtual Machine (VM)
Mức độ Ảo hóa	Ảo hóa cấp Hệ điều hành (OS Level).	Ảo hóa cấp Phần cứng (Hardware Level).
Hệ điều hành (OS)	Chia sẻ chung Kernel của Host OS.	Mỗi máy ảo chạy một Guest OS riêng biệt.
Dung lượng (Size)	Siêu nhẹ (vài MB đến vài trăm MB).	Nặng (vài GB đến hàng chục GB).
Thời gian Khởi động	Vài miligiây đến vài giây.	Vài phút (phải boot lại toàn bộ OS).
Hiệu năng (Performance)	Gần như Native Performance (Overhead cực thấp).	Có độ trễ nhất định do lớp Hypervisor.
Độ Cách ly (Isolation)	Cách ly cấp Tiến trình (Yếu hơn VM, cần cấu hình Security cẩn thận).	Cách ly cấp Phần cứng (Rất mạnh, an toàn cao hơn).
Công nghệ tiêu biểu	Docker, Podman, containerd, LXC.	VMware vSphere, KVM, Hyper-V, VirtualBox.

Ảnh container và kho lưu trữ ảnh

- **Container Image** là một tập tin tĩnh (Static File), bất biến (Immutable), chứa toàn bộ mã nguồn ứng dụng, các thư viện phụ thuộc (Libraries/Dependencies), tệp cấu hình và môi trường thực thi (Runtime) cần thiết để ứng dụng có thể khởi chạy.
- Container là tiến trình đang chạy được tạo từ image.
- Registry lưu trữ và phân phối image cho môi trường triển khai.
- Cần quét lỗ hổng image và tránh lưu bí mật trong image.



Kiến trúc tổng quan từ Image trong Registry đến Container thực thi.



Nguồn: Microsoft Learn / Docker containers, images, and registries - .NET | Microsoft Learn

CaaS là gì?

- Container as a Service cung cấp môi trường chạy và điều phối container.
- Người dùng quản lý ứng dụng container; nền tảng hỗ trợ cluster, scheduling, scaling và networking.
- CaaS thường gắn với Kubernetes hoặc dịch vụ Kubernetes được quản lý.
- Phù hợp khi hệ thống có nhiều microservices hoặc cần triển khai nhất quán nhiều môi trường.

CaaS là gì?

- Nói một cách ngắn gọn, CaaS nằm ở khoảng giữa IaaS và PaaS. Trong mô hình này, nhà cung cấp dịch vụ Cloud sẽ quản lý toàn bộ hạ tầng phần cứng, mạng và công cụ điều phối container (Orchestration Engine như Kubernetes), còn người dùng chỉ cần tập trung vào việc quản lý, đóng gói và triển khai các Container của ứng dụng.

Kubernetes ở mức khái quát

- Kubernetes là nền tảng điều phối container.
- Tự động triển khai, mở rộng, cập nhật và phục hồi container.
- Làm việc theo mô hình khai báo: mô tả trạng thái mong muốn, hệ thống tự điều chỉnh.
- Là nền tảng quan trọng cho cloud-native, nhưng có độ phức tạp vận hành cao.

Khi nào nên chọn container/CaaS?

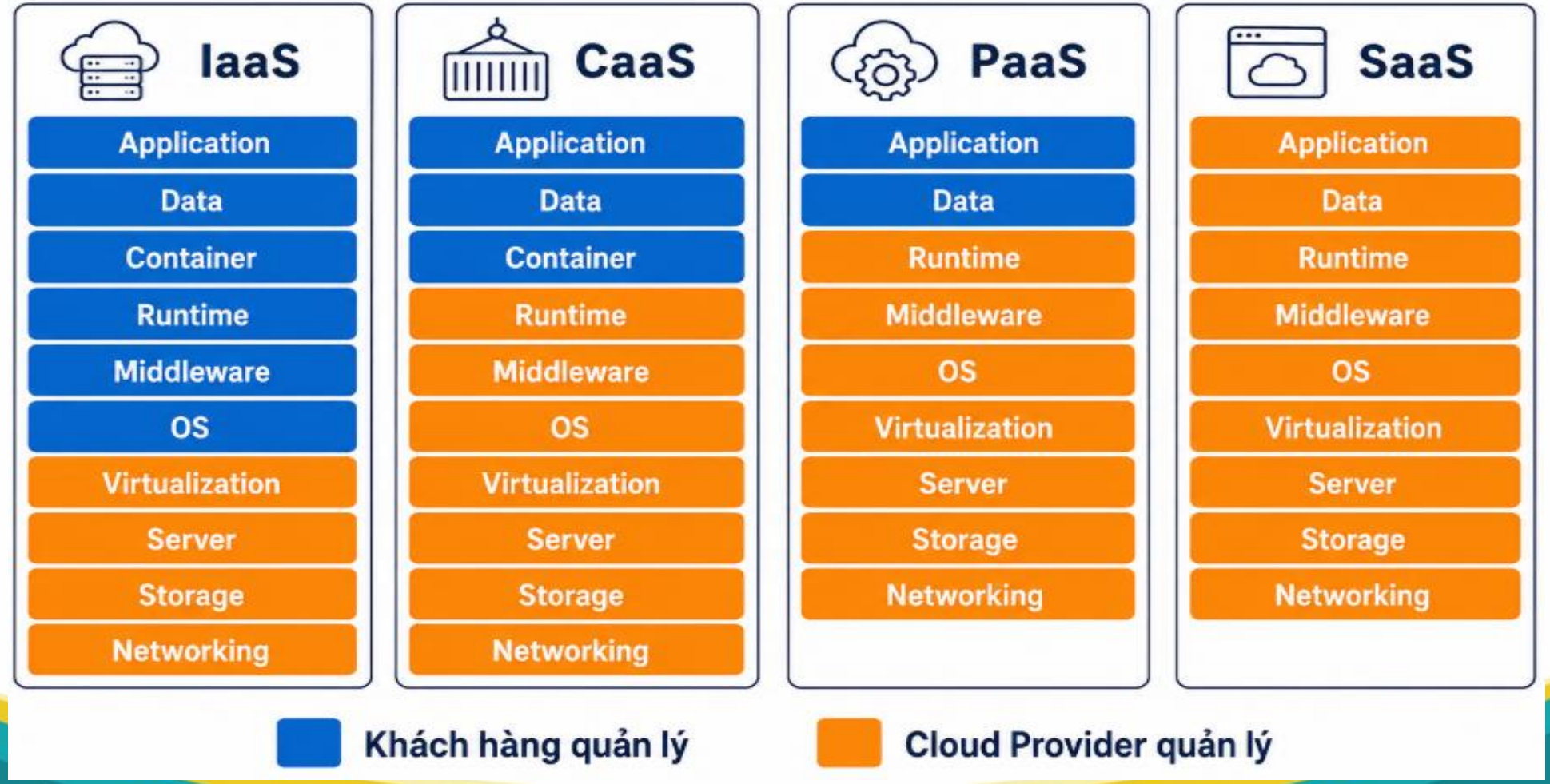
- Ứng dụng gồm nhiều dịch vụ nhỏ cần triển khai độc lập.
- Cần cùng một gói chạy trên dev, test và production.
- Cần rolling update, rollback và autoscaling ở mức dịch vụ.
- Đội ngũ có năng lực vận hành container và pipeline.
- Không nên chọn Kubernetes chỉ vì “hiện đại” nếu hệ thống nhỏ và đơn giản.

Các Dịch vụ CaaS Tiêu biểu trên Thị trường

Hiện nay, tất cả các nhà cung cấp Cloud lớn đều cung cấp CaaS dưới dạng **Managed Kubernetes**:

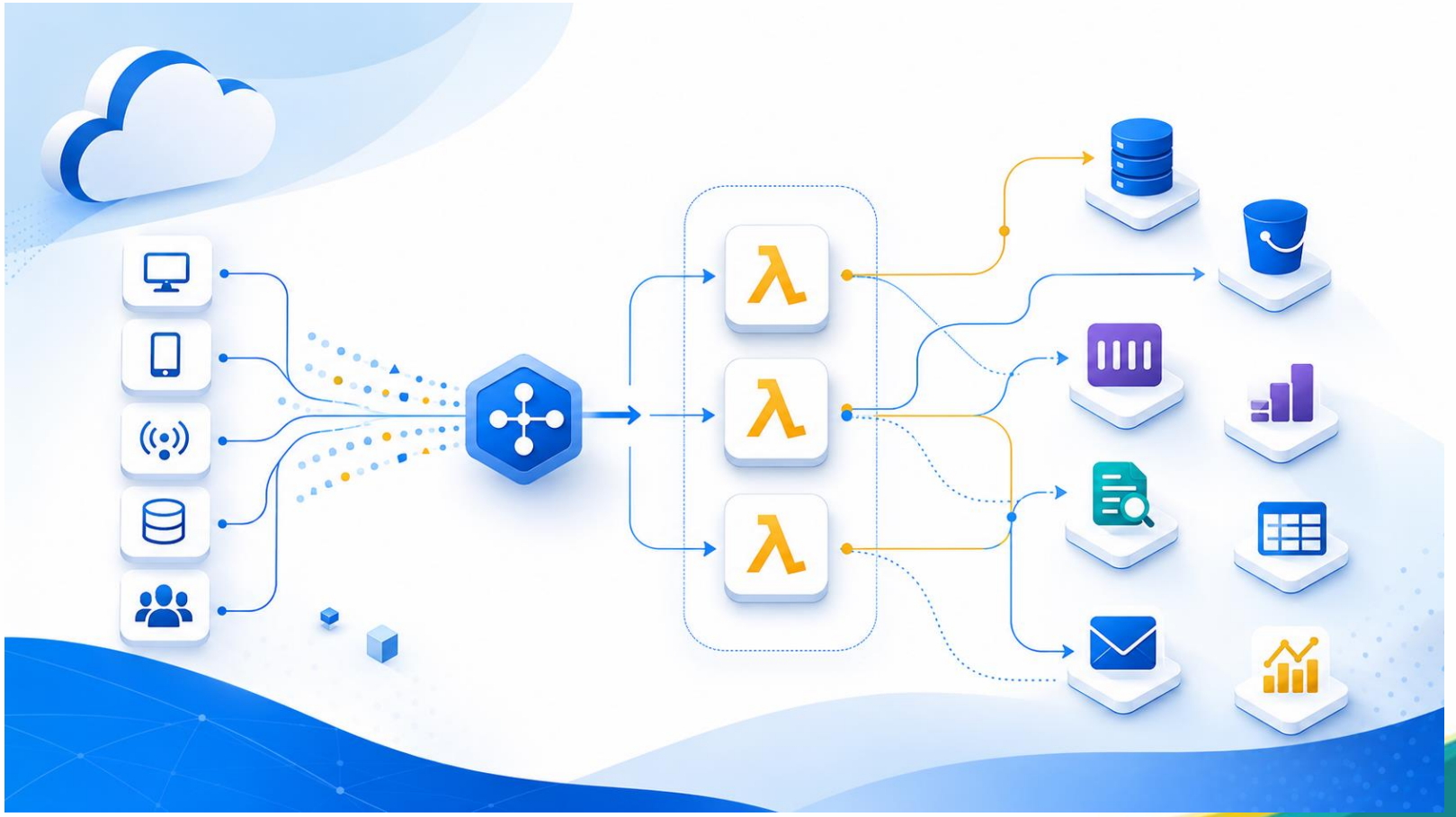
Nhà cung cấp	Tên dịch vụ CaaS (Managed K8s)	Serverless Container Engine (CaaS nâng cao)
AWS	**Amazon EKS** (*Elastic Kubernetes Service*)	**AWS Fargate** (Chạy Container không cần quản lý Node)
Google Cloud	**GKE** (*Google Kubernetes Engine*)	**Google Cloud Run**
Microsoft Azure	**AKS** (*Azure Kubernetes Service*)	**Azure Container Apps**
On-Premises / Multi-Cloud	**Rancher**, **Red Hat OpenShift**, **VMware Tanzu**	

Phân chia tránh nhiệm trong mô hình Cloud



Serverless và FaaS

- Serverless cho phép chạy mã theo sự kiện mà không trực tiếp quản lý máy chủ.



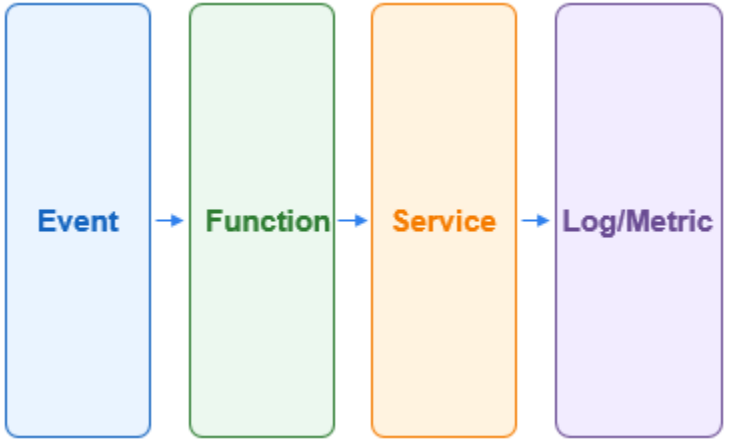
Kiến trúc đám mây đang hoạt động

Serverless là gì?

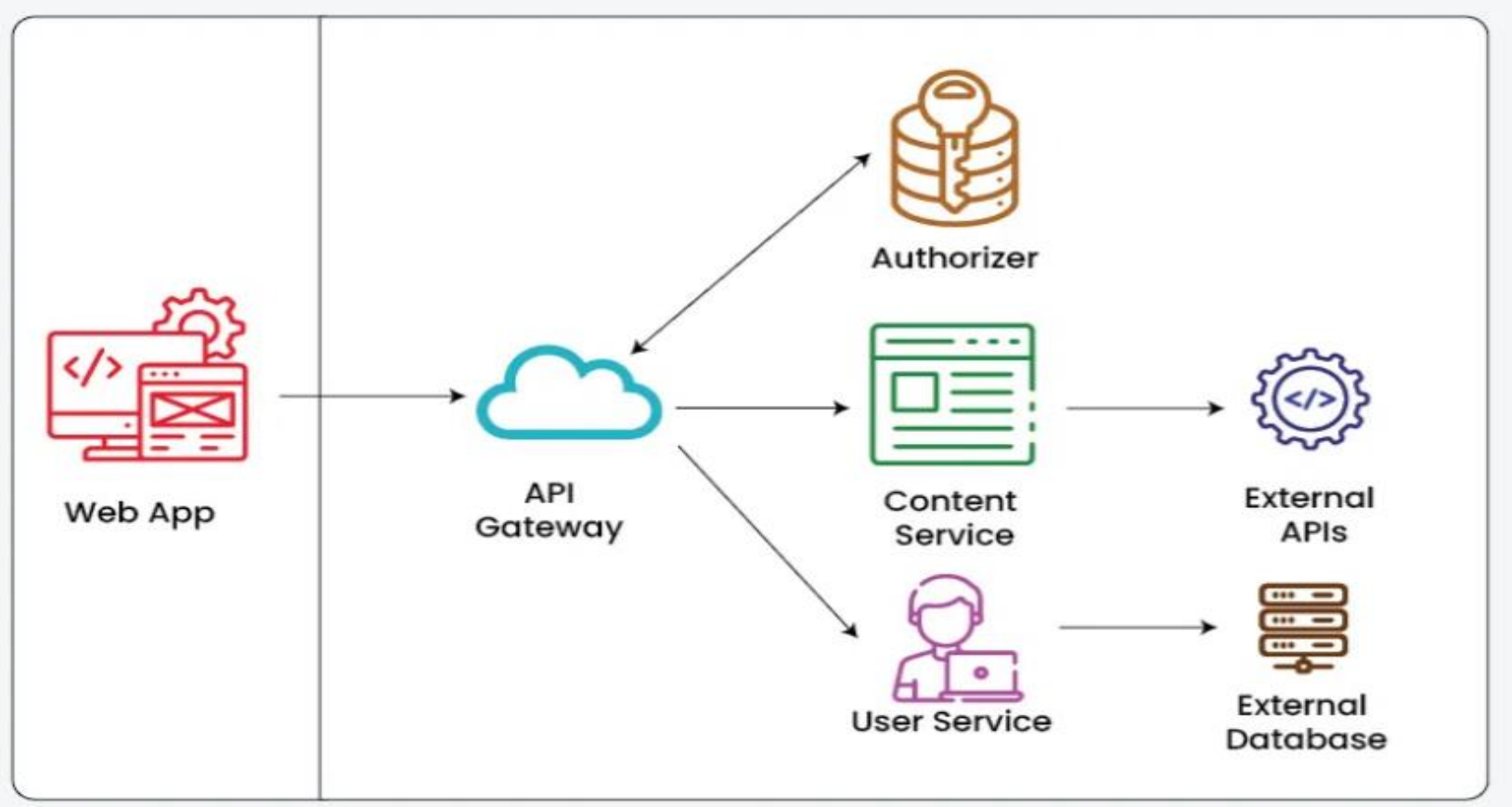
- Serverless là mô hình trong đó nhà cung cấp quản lý hạ tầng chạy ứng dụng ở mức cao.
- Người dùng tập trung vào hàm, sự kiện, dữ liệu và luồng xử lý.
- Tài nguyên thường tự động mở rộng theo số lượng sự kiện hoặc yêu cầu.
- Không có nghĩa là không có máy chủ; chỉ là người dùng không trực tiếp quản lý máy chủ.

FaaS: Function as a Service

- FaaS chạy các hàm nhỏ theo sự kiện kích hoạt.
- Mỗi hàm thực hiện một nhiệm vụ cụ thể: xử lý ảnh, gửi email, ghi log, xử lý webhook.
- Thời gian chạy thường ngắn, stateless và tự mở rộng.
- Tính phí thường dựa trên số lần gọi, thời gian chạy và tài nguyên sử dụng.



Kiến trúc ứng dụng Serverless dựa trên FaaS và API Gateway



Nguồn: GeeksforGeeks / Function as a Service (Faas) - System Design - GeeksforGeeks

Bài thực hành 2.3: serverless đơn giản

- Tạo một hàm serverless xử lý HTTP request hoặc sự kiện upload file.
- Cấu hình quyền truy cập tối thiểu cho hàm.
- Ghi log và xem kết quả thực thi.
- Thử thay đổi đầu vào để quan sát số lần gọi và thời gian chạy.
- Xóa tài nguyên sau khi hoàn thành bài lab.

Mục tiêu bài thực hành

1. Khởi tạo bảng dữ liệu NoSQL hoàn toàn Serverless với **Amazon DynamoDB.**
2. Viết và đóng gói một Hàm xử lý logic (**AWS Lambda**) bằng Python / Node.js.
3. Thiết lập **Amazon API Gateway** đóng vai trò điểm tiếp nhận HTTP Request.
4. Phân quyền an toàn giữa các dịch vụ Serverless bằng **IAM Role.**
5. Kiểm thử toàn bộ luồng hoạt động **(End-to-End Test**).

FaaS

- Nếu như trong mô hình PaaS hay CaaS, chúng ta vẫn phải duy trì các Server/Container chạy liên tục (24/7) để sẵn sàng phục vụ các yêu cầu, thì FaaS trừu tượng hóa hoàn toàn hạ tầng bên dưới. Với FaaS, lập trình viên chỉ cần tải lên từng khối mã nguồn xử lý logic độc lập (Hàm / Function). Hệ thống Cloud chỉ khởi chạy hàm đó khi có sự kiện (Event-driven) kích hoạt và tự động hủy bỏ ngay khi xử lý xong.

FaaS

Mã nguồn FaaS không chạy thường trực. Nó nằm ở trạng thái "nghỉ" cho đến khi nhận được một tín hiệu kích hoạt **(Event Trigger)**:

1. **Sự kiện xuất hiện (Event Triggered):** Người dùng gửi một HTTP Request qua API Gateway, tải một file ảnh lên S3 Bucket, hay thêm một bản ghi vào Database.
2. **Khởi tạo môi trường (Container Lifecycle):** Nền tảng FaaS tự động cấp phát một môi trường container siêu nhỏ (MicroVM / Ephemeral Container) và nạp hàm vào thực thi.
3. **Thực thi & Trả kết quả:** Hàm tính toán, xử lý logic, ghi dữ liệu và trả kết quả về cho client.
4. **Giải phóng tài nguyên:** Sau khi hoàn thành công việc (hoặc hết thời gian timeout), môi trường thực thi sẽ bị tiêu hủy hoàn toàn để giải phóng tài nguyên.

Các Dịch vụ FaaS Phổ biến nhất

Nhà cung cấp Cloud	Tên dịch vụ FaaS	Ngôn ngữ hỗ trợ phổ biến
AWS	**AWS Lambda**	Node.js, Python, Java, Go, C#, Ruby, Custom Runtime
Google Cloud	**Google Cloud Functions**	Node.js, Python, Go, Java, Ruby, PHP
Microsoft Azure	**Azure Functions**	C#, Node.js, Python, Java, PowerShell
Open Source / On-Prem	**OpenFaaS**, **Knative**	Mọi ngôn ngữ (chạy qua Docker Container)

Kiến trúc hướng sự kiện

- Sự kiện có thể đến từ HTTP request, queue, storage, database hoặc lịch định kỳ.
- Event bus hoặc message queue giúp tách rời các thành phần hệ thống.
- Hàm xử lý sự kiện và gọi các dịch vụ cloud khác.
- Kiến trúc này phù hợp xử lý bất đồng bộ và tải biến động.

Ưu điểm và hạn chế của serverless

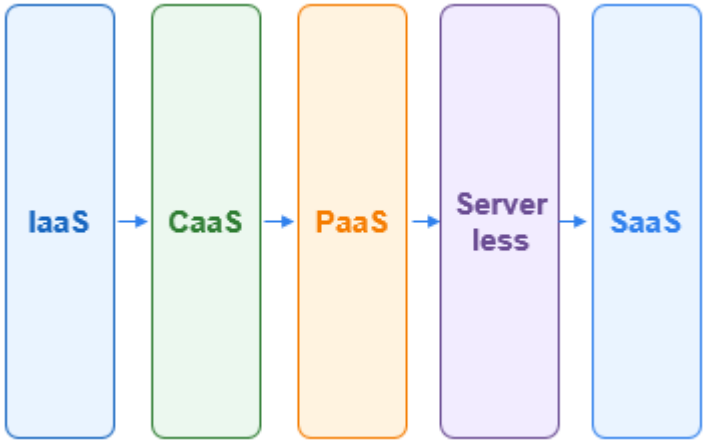
ƯU ĐIỂM CỦA SERVERLESS	HẠN CHẾ CỦA SERVERLESS
1. Không tốn công quản trị Hạ tầng & OS 2. Tối ưu chi phí tuyệt đối (Tính phí theo ms) 3. Tự động co giãn siêu tốc (Instant Scaling) 4. Tăng tốc độ đưa Sản phẩm ra Thị trường 5. Tích hợp mặc định Tính năng Kháng lỗi (HA)	1. Độ trễ từ hiện tượng "Cold Start“ 2. Giới hạn Thời gian Chạy & Tài nguyên 3. Khó khăn trong Debug & Testing tại Local 4. Nguy cơ Khóa chặt Nhà cung cấp (Vendor Lock) 5. Rủi ro Chi phí vượt kiểm soát khi Tải lớn

Bảng Cân nhắc Ra Quyết định (Decision Matrix)

Tiêu chí	NÊN CHỌN Serverless khi	NÊN CHỌN IaaS / CaaS / Container khi
Đặc điểm Lưu lượng (Traffic Pattern)	Lượng truy cập biến động thất thường, có nhiều khoảng thời gian "nhàn rỗi" (Idle time).	Lưu lượng ổn định, liên tục và có thể dự báo trước với tải cao (24/7).
Thời gian Xử lý (Duration)	Các tác vụ ngắn, xử lý tức thì (vài giây đến vài phút).	Các tác vụ tính toán kéo dài, chạy background job không giới hạn thời gian.
Mức độ Nhạy cảm Độ trễ	Chấp nhận độ trễ nhỏ khi khởi tạo (hoặc có ngân sách bật *Provisioned Concurrency*).	Yêu cầu độ trễ cực thấp, phản hồi mượt mà ổn định từng millisecond.
Quy mô Đội ngũ Ops	Đội ngũ IT/DevOps mỏng, muốn tối ưu vận hành.	Có đội ngũ System Admin/DevOps chuyên nghiệp quản trị cụm Kubernetes/Linux.

Mức độ kiểm soát và trách nhiệm

- IaaS: kiểm soát cao nhưng trách nhiệm vận hành lớn.
- PaaS: giảm vận hành máy chủ nhưng phụ thuộc nền tảng hơn.
- SaaS: dùng nhanh nhất nhưng tùy biến và kiểm soát thấp nhất.
- CaaS: linh hoạt với container nhưng cần kỹ năng vận hành cluster.
- Serverless: rất tự động hóa nhưng cần thiết kế theo sự kiện và stateless.



Tác động chi phí theo từng mô hình

- IaaS có thể phát sinh chi phí khi tài nguyên chạy liên tục nhưng không dùng hết.
- PaaS giảm chi phí vận hành nhưng có giá theo nền tảng và dịch vụ đi kèm.
- SaaS dễ dự toán theo người dùng nhưng tăng chi phí khi số tài khoản lớn.
- Serverless hiệu quả với tải biến động nhưng có thể đắt nếu chạy liên tục hoặc thiết kế sai.
- CaaS cần tính thêm chi phí vận hành cluster và năng lực đội ngũ.



Trách nhiệm bảo mật ở các mô hình dịch vụ

- Càng tự quản lý nhiều, trách nhiệm bảo mật của khách hàng càng lớn.
- IaaS: người dùng phải quản lý OS, patch, firewall, ứng dụng và dữ liệu.
- PaaS/CaaS: giảm một phần trách nhiệm hạ tầng nhưng vẫn cần bảo mật mã, cấu hình, secret.
- SaaS: trọng tâm là quản trị tài khoản, quyền truy cập, dữ liệu và cấu hình.
- Mô hình chia sẻ trách nhiệm sẽ được học sâu ở Chương 4.

IaaS: nhiều trách nhiệm khách hàng

PaaS/CaaS: chia sẻ nhiều hơn

SaaS: tập trung vào dữ liệu & quyền

Thank you!

